DISCLAIMER: This report and the pipeline it documents are a research and educational case study. They do not constitute investment, financial, legal, or tax advice, and no figure, risk categorization, or conclusion in this report or produced by the accompanying software should be relied upon for any investment, credit, or financial decision without independent verification against the original SEC filing and consultation with a qualified professional.

CHAPTER 1: INTRODUCTION

1.1 Industry and Organizational Context

This project sits within the financial services industry, in the domain of equity and credit research where analysts, portfolio managers, and corporate finance teams depend on the periodic disclosures that publicly listed companies file with securities regulators. Every U.S. listed company must file annual and quarterly reports, Form 10-K and Form 10-Q, with the SEC through its EDGAR system — the primary, legally attested source of a company's financial performance and the backbone of virtually every downstream activity in equity research and credit analysis. The sub-domain this project addresses is the application of LLMs and NLP to reading and structuring these filings, an area that has grown rapidly as disclosure volumes have outpaced what analyst teams can review line by line.

The operational environment reflects how a small analytics team would realistically approach this today: a Python pipeline downloads a filing from EDGAR, isolates the MD&A and Financial Statements sections, and passes each to GPT-4o for structured extraction. Output is stored as structured JSON alongside the raw text, and a Streamlit review application sits on top so a user can browse extracted metrics, inspect flagged risk language, compare periods, and run a fresh analysis from a browser. This is the kind of lean, tool-assisted workflow a boutique research team or individual analyst would use to bring LLM automation into an otherwise manual review process, not a large enterprise research desk.

It is important to frame this project as a decision-support tool rather than a replacement for analyst judgment. The industry treats disclosure data as high-stakes: a misread figure or overlooked risk disclosure can materially change an investment thesis. Any system built on a probabilistic language model rather than deterministic parsing has to be designed around verifiability from the outset, and that framing shapes nearly every design decision in the chapters that follow.

1.2 Background of the Business Problem

The business problem begins with a simple fact: a large company's annual report routinely runs over a hundred pages, and the two sections that matter most for analysis, MD&A and Financial Statements, are themselves long, inconsistently formatted, and interspersed with tables and cross-references. An analyst pulling even a small set of standard figures has to locate each

individually, confirm period and unit, and repeat the exercise every filing cycle — manual effort that scales linearly with little room for rule-based automation, since filers do not share one consistent layout.

A second, less visible layer is the qualitative language in the MD&A, where companies describe risk in deliberately hedged terms — "could," "may," "is subject to" — both for legal reasons and because genuine uncertainty is often the honest position. Tracking how risk posture shifts period to period requires a reader to sit with the full narrative and categorize each mention consistently, a subjective, time-consuming task that is hard to standardize across analysts.

LLMs are an obvious candidate to automate both halves, but naively applying one introduces a more serious risk: hallucination — a plausible-looking figure that never appears in the source text, or a silent unit conversion that corrupts every downstream comparison. The real business problem is not simply whether an LLM can read a 10-K, but whether an extraction pipeline can be fast, comparable across periods, and verifiable enough to trust where an unverified error has real financial consequences.

1.3 Significance of the Study

The practical significance lies in showing that the manual burden of first-pass filing review can be meaningfully reduced without giving up traceability. Requiring a verbatim source snippet for every non-null value, and programmatically checking it before ever presenting output, means a reviewer can spot-check in seconds rather than rereading the filing — the same discipline applies to risk-language extraction, quoting rather than paraphrasing so a reviewer can quickly agree or disagree with a categorization.

Beyond the immediate benefit, the study carries significance for how organizations deploy language models in high-stakes, numerically precise domains generally. Constraining the model to a narrow metric set, forcing null over guessed values, and mechanically verifying every citation are techniques transferable to other document-heavy, low-tolerance-for-error settings such as legal contract review or compliance reporting.

Finally, the study has academic value in its own right: it combines an end-to-end NLP pipeline with a rigorous evaluation methodology, reporting accuracy honestly including failure cases rather than only favorable results — the evaluation-first orientation the literature reviewed in the next chapter calls for.

CHAPTER 2: REVIEW OF LITERATURE AND METHODOLOGY

2.1 Review of Relevant Studies

Text analysis of corporate disclosures predates the current wave of large language models by over a decade, and this older literature still shapes how a project like this one should think about risk. Loughran and McDonald (2011) showed that generic sentiment dictionaries misclassify ordinary financial and legal vocabulary as negative, arguing that filing analysis needs domain-specific word lists rather than off-the-shelf sentiment tools. Kogan et al. (2009) demonstrated that 10-K text, reduced to simple bag-of-words features, predicts post-filing stock return volatility, an early indication that MD&A narrative is informative rather than mere boilerplate. Li (2010) applied a naive Bayesian classifier to forward-looking statements in annual reports and found their tone has measurable predictive power for future earnings, part of the motivation for treating risk-language extraction as a first-class task here rather than a secondary one.

A second body of work builds the datasets that make financial NLP tractable at scale. Malo et al. (2014) released the Financial PhraseBank, a standard sentiment benchmark drawn from financial news. More directly relevant, Loukas et al. (2021) built EDGAR-CORPUS from SEC filings and showed filing-pretrained models outperform general-purpose ones on finance tasks, supporting this project's decision to work with EDGAR filings directly. The same group's FiNER (Loukas et al., 2022) frames numeric-entity recognition for XBRL tagging as a structured extraction problem conceptually close to this project's metric-extraction module, though as token-tagging rather than LLM prompting. Chen et al. (2021) introduced FinQA, pairing filing excerpts with questions requiring multi-step numerical reasoning, and Zhu et al. (2021) followed with TAT-QA, a hybrid tabular-and-textual benchmark. Both make the point this project relies on: extracting a number correctly is only half the task, since verifying it is grounded in the source is what separates a trustworthy system from a plausible-sounding one.

A third strand covers domain-adapted, finance-specific language models. Araci (2019) and Yang et al. (2020) each produced a FinBERT variant fine-tuned or pretrained on financial text, establishing that domain adaptation measurably improves finance-specific performance over general models. Wu et al. (2023) scaled this with BloombergGPT, a fifty-billion-parameter model trained on mixed general and proprietary financial text, and Xie et al. (2023) took a

complementary instruction-tuning approach with PIXIU. This project deliberately takes a different path, using general-purpose GPT-4o under a tightly constrained extraction prompt rather than a fine-tuned model, on the reasoning that prompt-level constraints plus mechanical output verification can match reliability on a narrow task without the cost of domain-specific pretraining, a trade-off revisited later in this chapter.

The most directly relevant strand concerns hallucination and factual verification, the central risk this project is designed around. Ji et al. (2023), in a widely cited hallucination survey, categorize errors as intrinsic (contradicting the source) or extrinsic (unverifiable against it), and argue that grounding output in retrievable source text is one of the few mitigation strategies with consistent empirical support, a principle applied here directly through snippet verification. Lewis et al. (2020) proposed retrieval-augmented generation for grounding output in retrieved passages, applied here in lightweight form by scoping input to one regex-isolated filing section. Kadavath et al. (2022) found language models are poorly calibrated when validating their own free-form generations, precisely why this project performs an independent, code-based citation check rather than trusting the model's confidence. Min et al. (2023) proposed FActScore, decomposing generated text into atomic facts verified independently against a trusted source, a philosophy mirrored at smaller scale in this project's per-metric and per-risk-mention verification.

Taken together, this literature supports two conclusions that shaped this project directly. First, financial disclosure text is distinct enough that generic NLP tools do not transfer cleanly, which is why the metric list, risk taxonomy, and section-boundary logic here were built around 10-K/10-Q structure rather than borrowed generically. Second, the difference between an impressive LLM demo and a system an analyst can rely on is almost entirely a question of verification: whether every claimed number and risk statement traces back to source text mechanically, not on the model's word. The methodology below, and the evaluation in Chapter 4, are built around closing exactly that gap.

2.2 Problem Statement

The problem this project addresses can be stated concretely as follows: given a company's SEC 10-K or 10-Q filing, existing approaches to extracting standardized financial metrics and identifying risk disclosures are either slow and manual, requiring an analyst to read and cross-reference dense filing text by hand, or fast but unreliable, relying on brittle rule-based parsing that breaks across filers with different formatting conventions, or on naive large language

model prompting that carries a real risk of hallucinating figures that were never stated in the source document. There is, in other words, a gap between the speed that large language models offer and the trust that financial analysis demands, and this project sets out to close that gap for a bounded but representative task: extracting eight standard financial metrics and categorizing risk language from the MD&A section, with every extracted item mechanically traceable back to the filing it came from, and with the resulting accuracy measured honestly against a human-built ground truth rather than assumed.

2.3 Need for the Study

The need for this study follows directly from how quickly large language models have moved from research demonstrations into production financial workflows, often faster than the tooling needed to verify their output has matured alongside them. Investment research teams, credit analysts, and corporate finance functions are all under pressure to process a growing volume of disclosure text without a proportional increase in headcount, and generative AI is the most visible candidate for closing that gap. At the same time, regulators and internal risk functions are increasingly asking not just whether an AI system produces useful output, but whether that output can be audited, a question most publicly discussed LLM finance demonstrations do not answer convincingly because they report accuracy on curated benchmarks rather than on the messy, inconsistently formatted filings analysts actually work with.

There is accordingly a need for concrete, reproducible case studies that go beyond reporting a single accuracy number and instead document the full pipeline, including where it fails and why. This project responds to that need by treating evaluation and failure analysis as first-class outputs rather than an afterthought, building a pipeline on a single, well-understood company across two fiscal years before making any claim about generalizing further, and reporting every metric that fails to match ground truth alongside every one that succeeds. This disciplined, narrow-scope approach is itself a response to a pattern the literature review above surfaces repeatedly: systems that look impressive in aggregate often hide meaningful failure rates in the specific cases that matter most, and the only way to know whether that is true here is to measure it directly.

2.4 Scope of the Study

The scope of this study is deliberately bounded rather than exhaustive. It covers two SEC filing types, Form 10-K annual reports and Form 10-Q quarterly reports, and restricts financial metric

extraction to eight core figures that are consistently reported across most filers: total revenue, net income, diluted earnings per share, gross profit, operating income, total assets, total liabilities, and operating cash flow. Risk-signal extraction is similarly bounded to eight fixed categories spanning macroeconomic conditions, supply chain, competition, regulatory and legal exposure, cybersecurity and data privacy, foreign exchange, litigation, and labor, chosen to keep any resulting risk-frequency comparison readable rather than attempting to capture every possible risk theme a company might mention.

In terms of empirical validation, the study follows the principle of proving the approach correctly on one company before attempting to generalize it: it is validated first on Apple Inc. (ticker AAPL) across its fiscal year 2023 and fiscal year 2024 10-K filings, then extended, as section 4.4 documents, to four additional filers spanning three further sectors — Johnson & Johnson, Costco (both a 10-K and, live, a 10-Q), Delta Air Lines, and Etsy — with the pipeline's live-run capability further demonstrated on additional periods beyond those formally scored. The study still does not extend into portfolio construction, valuation modeling, or any form of investment recommendation; its scope stops at producing verified, structured, and comparable disclosure data for a human analyst to use in their own downstream judgment.

2.5 Objectives of the Project

The project was guided by a small set of concrete objectives that together define what a successful outcome looks like. The first objective was to build an automated pipeline capable of retrieving a company's 10-K or 10-Q filing directly from SEC EDGAR and isolating the Management's Discussion and Analysis and Financial Statements sections from the surrounding filing text without manual intervention. The second was to extract a fixed set of standardized financial metrics from the Financial Statements section using a large language model, in a way that is explicit about the unit each figure is reported in and refuses to guess a value that is not explicitly stated in the source text. The third objective was to extract and categorize risk-related language from the MD&A section using a fixed, comparable taxonomy, quoting rather than paraphrasing the underlying disclosure so a human reviewer can quickly validate the categorization. The fourth objective was to build mechanical, code-based verification for both of the above, checking that every claimed source citation actually appears in the filing text it is attributed to, so that hallucinated figures or quotes are caught automatically rather than relying on manual spot-checking. The fifth objective was to compute period-over-period comparisons of extracted metrics that correctly flag missing data and unit mismatches

rather than producing a misleading percentage change, and to visualize both the resulting trends and the risk-mention frequency in a form suitable for review. The sixth and final objective was to evaluate the entire pipeline's output against a manually constructed ground truth and to report accuracy, including every failure case, as an honest measure of how much a system like this can currently be trusted.

2.6 Hypotheses

Given the project's central concern with reliability rather than raw automation, the guiding hypotheses are framed around extraction accuracy against ground truth rather than around any predictive or investment-related claim. The null hypothesis, H0, states that the LLM-based extraction pipeline, when constrained by an explicit no-hallucination instruction and verified mechanically against source text, does not achieve financial metric extraction accuracy that is reliable enough to be useful to a human analyst, meaning its output would still require the analyst to re-verify most figures against the original filing before use. The alternative hypothesis, H1, states that the pipeline does achieve accuracy reliable enough to meaningfully reduce, though not eliminate, the analyst's manual verification burden, evidenced by a high rate of exact matches against a manually built ground truth and by a low rate of unverified or flagged source citations.

These hypotheses are evaluated directly in Chapter 4 by comparing the pipeline's extracted metrics for Apple's fiscal year 2023 and fiscal year 2024 10-K filings against a manually constructed ground truth using the normalized exact-match methodology described later in this chapter, with the accuracy percentage and the specific failure cases, if any, reported without adjustment.

2.7 Research Design and Methodology

This project follows an applied, design-and-evaluate research methodology rather than a purely theoretical one: a working system was built first, then evaluated empirically against a ground truth constructed independently of the system's own output, so that the evaluation results in Chapter 4 are not circular. The end-to-end workflow follows six stages, described in turn below, each implemented as a separate, independently runnable module so that a failure or a needed adjustment at one stage does not require rebuilding the others.

Data Sourcing

Filing data is sourced directly from SEC EDGAR rather than from any secondary or aggregated dataset, using the sec-edgar-downloader Python package alongside direct calls to EDGAR's submissions API. Because sec-edgar-downloader's default behavior only retrieves the most recently filed document of a given form type, the pipeline first resolves the exact accession number corresponding to the requested fiscal period end date through EDGAR's submissions JSON endpoint, then constrains the download to a narrow date window around that filing's actual filed date. This two-step lookup exists specifically to prevent a subtle but consequential error: silently downloading the same latest filing twice when two different historical periods were requested. Every EDGAR request carries a descriptive User-Agent with a real contact email, as SEC's access policy requires, configured through an environment variable rather than hard-coded into the pipeline.

Preprocessing and Segmentation

The raw filing is downloaded as HTML and converted to plain text using BeautifulSoup with an lxml parser, stripping script and style tags while preserving paragraph breaks. The resulting full-text document is then segmented using a tolerant, regular-expression-based boundary detector that locates the start of the Management's Discussion and Analysis section (Item 7 for a 10-K, Item 2 for a 10-Q) and the start of the Financial Statements section (Item 8 for a 10-K, Item 1 for a 10-Q), each validated against a required title phrase appearing within a short window after the candidate match. This title-validation step exists because a bare line-start match on "Item 7" or "Item 8" is not sufficient on its own: several large filers, Microsoft's 10-K among them, repeat the bare item number as a running page header throughout the section, which would otherwise cause a naive boundary detector to select an arbitrary page deep inside the section rather than its true starting header. This preprocessing stage is also where the project enforces the rule of never handing the language model more than one filing section at a time, since passing an entire multi-hundred-page 10-K to the model in one call would both exceed a reasonable context budget and make it far harder to constrain what the model is allowed to talk about.

Model Selection and Prompting Strategy

Exhibit 2.1 — Verbatim System Prompts
The complete system prompt text sent to GPT-4o for each extraction task, reproduced exactly as it appears in the source code (src/extract_metrics.py and src/extract_risks.py) with only the {metrics} / {categories} placeholder substituted at call time from the METRICS and RISK_CATEGORIES lists shown in Chapter 3, is given below.
extract_metrics.py — SYSTEM_PROMPT

```
You are a financial data extraction engine. You will be given raw text extracted from
the Financial Statements section of an SEC filing (10-K or 10-Q).
Extract ONLY the following metrics: {metrics}
Rules (violating these is worse than reporting a metric as not found):
1. NEVER estimate, calculate, or infer a number that is not explicitly stated in the
text. If a metric is not present verbatim, set "value": null and "source_snippet":
null.
2. For every non-null value, include a "source_snippet": a short, EXACT, verbatim
excerpt (under 15 words) copied character-for-character from the provided text that
contains that number. This is used to programmatically verify you did not
hallucinate.
3. Report "unit" exactly as presented in the filing (e.g. "thousands", "millions",
"actual"). Do not convert units yourself.
4. If a figure appears to be a restated or revised prior-period number (common in 10-
Ks that show current + prior year side by side), extract the value for the period
explicitly requested and note "restated": true if the text flags it as such (e.g. "as
restated", "as revised").
5. Respond with ONLY valid JSON, no markdown fences, no commentary.
Output schema:
{"metrics": [{"metric": "<name from the list above>", "value": <number or null>,
"unit": "<string or null>", "restated": <true/false>, "source_snippet": "<verbatim
excerpt or null>"}]}
```

extract_risks.py — SYSTEM_PROMPT

```
You are analyzing the MD&A (Management Discussion & Analysis) section of an SEC
filing to identify risk signals the company itself discusses.
Categories to use (use ONLY these, do not invent new ones): {categories}
For each distinct risk statement you find:
1. Quote the actual sentence (or a tight excerpt, under 30 words) — do not
paraphrase.
2. Assign exactly one category from the list above (pick the closest fit).
3. Rate severity_language as "explicit" (company states clear negative
impact/concern) or "hedged" (forward-looking boilerplate, conditional language like
"could" / "may").
Do not invent risks that aren't discussed. If the MD&A section barely discusses a
category, that's a valid finding — do not pad the list to cover every category.
Respond with ONLY valid JSON:
{"risk_mentions": [{"category": "<category>", "excerpt": "<quote>",
"severity_language": "explicit|hedged"}]}
```

The project uses OpenAI's GPT-4o as its extraction model, accessed through the standard chat completions API with the response format constrained to JSON and the sampling temperature set to zero to maximize determinism across repeated runs. A domain-adapted, fine-tuned model such as FinBERT or a finance-specific instruction-tuned model was considered, consistent with the literature reviewed earlier in this chapter, but a general-purpose frontier model under a tightly scoped system prompt was chosen instead, on the basis that the task at hand, extracting a small, fixed set of named figures and quoting risk language rather than open-ended financial reasoning, is well suited to prompt-level constraints and does not clearly benefit from the additional infrastructure of maintaining a separately fine-tuned model. The system prompt for metric extraction explicitly enumerates the eight target metrics, instructs the model to return null rather than estimate any value not explicitly stated in the text, requires a verbatim source snippet under fifteen words for every non-null value, and requires the unit to be reported exactly as presented in the filing rather than silently converted. The system prompt for risk extraction similarly constrains the model to a fixed eight-category taxonomy, requires a verbatim quote

rather than a paraphrase for every risk mention, and asks the model to rate each mention's severity language as either explicit or hedged.

Extraction and Verification

Each API call is wrapped in a retry policy using exponential backoff, tolerating transient network or rate-limit failures without silently dropping a filing section. Once the model returns its structured JSON response, every claimed source snippet is independently checked against the segmented section text using a whitespace-normalized substring match, entirely outside of the model's own control. A metric or risk mention whose claimed snippet cannot be found verbatim in the source text is flagged as unverified rather than silently accepted, giving the pipeline a mechanical, code-based hallucination check that does not depend on trusting the model's own assessment of its output, directly following the verification-over-self-assessment principle discussed in the literature review.

Period Comparison and Evaluation

Once two periods of extracted metrics exist for a company, a dedicated comparison module computes period-over-period percentage changes for each metric, explicitly flagging two edge cases rather than letting them produce misleading output: a metric missing from one period is reported as not comparable rather than treated as a hundred percent change, and a unit mismatch between periods, for instance a filer switching from reporting in thousands to reporting in millions, is flagged rather than allowed to feed into a nonsensical percentage calculation. Final evaluation is performed by an independent scoring module that compares extracted metrics against a manually constructed ground truth CSV, normalizing every value to a common base unit before comparison so that the evaluation is not fooled by superficial formatting differences such as commas, currency symbols, or unit labels. A match is scored only when the normalized extracted value and the normalized ground-truth value agree to within a small numerical tolerance, and every non-match is recorded with an explicit failure reason rather than being folded into a single opaque accuracy number.

2.8 Analytical Techniques

The analytical core of the pipeline is large language model-based structured information extraction, using OpenAI's GPT-4o under constrained, schema-enforced prompting rather than any classical supervised machine learning model, since the task requires reading unstructured narrative and tabular text rather than learning from labeled training examples. Supporting this

core extraction step, the pipeline uses BeautifulSoup and lxml for HTML parsing and text extraction, regular-expression-based pattern matching validated against title-phrase heuristics for section segmentation, and the tenacity library for retry-with-backoff resilience around external API calls. Post-extraction analysis relies on pandas for tabular manipulation of both extracted metrics and the ground-truth comparison data, and on plotly for all visualizations, including period-over-period trend lines, a risk-mention frequency heatmap across categories, and a horizontal bar chart of the largest flagged period-over-period deviations. The review and live-analysis interface is built with Streamlit, chosen for how quickly it allows a data-oriented interface to be built directly on top of the same Python pipeline functions used by the command-line runner, so that a live run triggered from the browser and a run triggered from the command line execute identical extraction logic rather than two parallel implementations that could drift apart.

Deliberately absent from this list is any classical machine learning model trained on labeled data, such as a fine-tuned classifier for risk categorization or a regression model for metric prediction. This is a considered choice rather than an oversight: the extraction and categorization tasks this project targets are well within the reach of a well-constrained large language model prompt, and introducing a separately trained model would add a data-labeling and model-maintenance burden without a clear accuracy benefit at the scale this project operates at. Where the literature reviewed earlier in this chapter shows domain-specific fine-tuning delivering a clear benefit, such as in large-scale financial sentiment classification, that benefit is most pronounced at a scale and volume of data well beyond what this project's single-company, few-period evaluation requires to demonstrate its core claim about verifiable extraction.

2.9 Limitations of the Study

This study carries several limitations that should be read alongside its results in later chapters rather than treated as afterthoughts. The empirical evaluation is conducted on a single company across two fiscal years, which is sufficient to validate the pipeline's approach in depth but is not, on its own, evidence that extraction accuracy will hold at the same level across the far more varied formatting conventions used by other filers; the segmentation module's boundary-detection logic in particular was hand-tuned against the specific structure of Apple's filings and explicitly documented in the code as needing filer-specific adjustment before being trusted on a company with a meaningfully different filing layout. The scope is further limited to eight

financial metrics and eight risk categories, chosen for consistency and readability rather than completeness, so the pipeline does not capture less commonly reported metrics or risk themes that fall outside the fixed taxonomy.

A second limitation concerns the source-snippet verification mechanism itself: it confirms that a claimed quote appears verbatim somewhere in the filing section, which catches outright hallucination effectively, but it cannot confirm that the model attached the correct number to the correct metric label when a snippet legitimately does exist in the text, nor can it detect a subtly wrong but textually-grounded interpretation. This is why the evaluation in Chapter 4 relies on an independently built ground truth rather than treating snippet verification alone as proof of correctness. A third limitation is dependence on a proprietary, externally hosted model: GPT-4o's behavior is not fully under the project's control, and its outputs, while constrained to zero temperature for determinism, are not guaranteed to be perfectly reproducible across API versions or over time, which is a general limitation of building an evaluated system on top of a commercial model rather than a fixed, locally controlled one. Finally, the study evaluates extraction accuracy and citation correctness but does not evaluate downstream decision quality, meaning it does not, and is not intended to, measure whether an analyst using this tool's output would make better investment or credit decisions than one working from the filing directly.

CHAPTER 3: DATA FRAMEWORK, VARIABLE DESCRIPTION AND ANALYTICAL MODEL APPLICATION

3.1 Problem Context and Functional Area Description

Reframed in technical terms, this project is not one homogeneous machine learning task but a pipeline of four functional areas, each with its own reliability profile. The first is document segmentation, a lightweight retrieval problem: locate and isolate the MD&A and Financial Statements spans from a long filing, discarding everything else before any model sees it. This is implemented deterministically through validated regular-expression matching rather than a learned retriever, a choice revisited in section 3.6.

The second and most central area is structured information extraction (slot filling): given the isolated Financial Statements text, identify eight named numeric entities and return each as a typed value with unit and source citation, blending named-entity recognition with constrained generation via a prompted LLM rather than a trained sequence-tagger. The third is text classification: risk sentences from the MD&A are assigned to one of eight fixed categories, zero-shot, by the same model. The fourth is deterministic quantitative computation — percentage-change calculation and ground-truth scoring — plain, auditable arithmetic with no learned component.

Distinguishing these matters because risk is not evenly distributed: segmentation and the final arithmetic are deterministic and repeatable, while extraction and classification carry genuine model-driven uncertainty, which is exactly why the verification mechanisms described later, and evaluated in Chapter 4, are concentrated there.

3.2 Project Environment and Constraints

The system was developed in a Python 3.10 virtual environment, keeping dependency versions (requirements.txt) separate from the host machine. No model training or fine-tuning occurs anywhere in the pipeline; every model call is a hosted inference request to GPT-4o, so the compute footprint is just a Python process making outbound calls to two external services, SEC EDGAR and the OpenAI API.

Table 3.1 — Core Software Dependencies

Package	Role in the Pipeline
sec-edgar-downloader	Retrieves raw 10-K / 10-Q filing HTML from SEC EDGAR for a given ticker and form type
openai	Client library for GPT-4o structured-extraction and risk-classification calls
beautifulsoup4 / lxml	Parses filing HTML and converts it into clean, paragraph-preserving plain text
pandas	Tabular handling of extracted metrics, comparisons, and ground-truth data
plotly / matplotlib	Generates trend charts, the risk-mention heatmap, and deviation charts
streamlit	Powers the browser-based review and live-analysis application
tenacity	Adds retry-with-exponential-backoff resilience around external API calls
python-dotenv	Loads API keys and contact-email configuration from a local .env file
pyrate-limiter	Keeps EDGAR request rates within SEC's fair-access policy

The extraction model is pinned to a specific dated snapshot, gpt-4o-2024-11-20, rather than the floating “gpt-4o” alias used in earlier development runs, so a reader can tie any reported result to a known, fixed model version instead of whatever OpenAI's alias happens to resolve to on a given day — the alias is not a stable snapshot and does move over time. The Chapter 4 results in this report were produced by a run against this pinned snapshot on 2026-08-05. Re-running the pipeline under this pinned snapshot, after an earlier development run under the unpinned alias, changed two observable outcomes without changing any underlying filing data: the one FY2023/FY2024 metric citation that fails verbatim source-snippet verification moved from the FY2024 filing to the FY2023 filing (same line item, Total Liabilities, same failure mode, a fabricated dollar sign), and the risk-mention counts in both periods changed slightly, including one category, Labor / Talent, appearing in FY2023 that the earlier run did not surface. Both are discussed with their actual figures in Chapter 4 and are treated here as evidence for, not against, the model-version caveat already raised in section 2.9: pinning a version documents what was run, it does not make that version's output perfectly stable on its own.

Several constraints shaped the build. SEC EDGAR requires a descriptive User-Agent with a real contact email on every request (via an environment variable, not hard-coded), rate-limited to EDGAR's fair-access guidelines. The OpenAI API imposes a cost and context-window constraint — every token is billed, and long inputs are harder to keep the model faithful to — which is why segmentation never hands the model more than one section at a time. Sampling temperature is fixed to zero for determinism, though this reduces rather than eliminates run-to-

run variation, since a hosted model's behavior is not fully within the project's control. Finally, Windows consoles default to a codepage that cannot render every character in real filing text (em dashes, currency symbols), requiring explicit UTF-8 stream reconfiguration to prevent mid-run crashes.

3.3 Dataset Profile

This project does not use a conventional labeled training dataset, since no component is trained or fine-tuned; GPT-4o runs entirely at inference time under a fixed prompt. What it does use is a small, purpose-built evaluation corpus of real SEC filings for one company, Apple Inc. (AAPL), paired with a manually constructed ground truth used exclusively to score output after the fact.

Table 3.2 — Filing Corpus Used in This Study

Ticker	Form	Fiscal Period End	Role in the Study
AAPL	10-K	2023-09-30	Earlier period (Period A) for period-over-period comparison; scored against ground truth
AAPL	10-K	2024-09-28	Later period (Period B) for period-over-period comparison; scored against ground truth
AAPL	10-Q	2026-06-27	Live-run demonstration of the pipeline on a quarterly filing via the Streamlit application

The ground truth was built by manually reading Apple's FY2023 and FY2024 10-K filings and recording the correct value and unit for each of the eight target metrics, sixteen observations in a CSV (company, period, metric, true_value, true_unit). It plays the role a labeled test set plays in supervised learning, except it scores an inference-time pipeline rather than training one, keeping Chapter 4's evaluation independent of anything the pipeline itself produced.

3.4 Variables and Constructs Description

Two structured output schemas define every variable the pipeline produces. The first, the extracted-metric record, is the unit of output from the financial-metrics extraction stage, and every field in it exists for a specific reason tied back to the reliability concerns discussed in Chapter 2.

Table 3.3 — Extracted Metric Record Schema

Field	Type	Description
metric	string	Name of the metric, constrained to one of the eight values in the fixed target list
value	number or null	The figure as reported in the filing; null if the metric was not explicitly stated
unit	string or null	Unit exactly as presented in the filing (thousands, millions, or actual), never converted at this stage
restated	boolean	True if the filing flags this figure as a restated or revised prior-period number
source_snippet	string or null	A short verbatim quote containing the value, used for mechanical verification
verified_in_source	boolean	Computed field: true only if source_snippet was found verbatim in the section text

The eight metric names that populate the metric field are themselves the core numeric constructs this study is built around, chosen because they are reported consistently across most filers and together give a compact but representative view of a company's financial position.

Table 3.4 — Financial Metric Constructs and Definitions

Metric	Definition	Primary Statement
Total Revenue	Aggregate net sales recognized during the reporting period, before cost of goods sold is deducted	Statement of Operations
Net Income	Bottom-line profit remaining after all operating expenses, interest, and taxes are deducted from revenue	Statement of Operations
Diluted EPS	Net income attributable to shareholders divided by diluted weighted-average shares outstanding, including dilutive securities	Statement of Operations
Gross Profit	Total revenue less cost of goods sold, before operating expenses are deducted	Statement of Operations
Operating Income	Gross profit less operating expenses such as R&D and SG&A, excluding non-operating items	Statement of Operations

Metric	Definition	Primary Statement
Total Assets	Sum of all current and non-current assets held at period end	Balance Sheet
Total Liabilities	Sum of all current and non-current obligations owed at period end	Balance Sheet
Operating Cash Flow	Net cash generated by core business operations during the period	Statement of Cash Flows

The second schema, the risk-mention record, is the unit of output from the MD&A risk-classification stage. Its constructs are qualitative rather than numeric, and the taxonomy of eight fixed categories was deliberately kept small enough that a resulting frequency table or heatmap across categories stays readable.

Table 3.5 — Risk Mention Record Schema and Category Taxonomy

Field / Category	Description
category	One of eight fixed categories, described below, assigned to the closest-fitting risk statement
excerpt	A verbatim quote (or tight excerpt) of the risk statement, never a paraphrase
severity_language	"Explicit" if the company states a clear negative impact, or "hedged" if the language is conditional (could/may)
Macroeconomic / Market Conditions	Broad economic factors such as inflation, interest rates, or demand cycles
Supply Chain	Sourcing, component availability, and manufacturing/logistics dependencies
Competition	Competitive pressure on pricing, market share, or product differentiation
Regulatory / Legal	Exposure to new laws, regulatory action, or compliance requirements
Cybersecurity / Data Privacy	Data breach risk, information-security incidents, and privacy obligations
Foreign Exchange / Currency	Exposure to currency fluctuation in international operations
Litigation	Ongoing or threatened legal proceedings against the company
Labor / Talent	Workforce availability, retention, and labor-cost pressure

The verified_in_source field deserves separate attention because, unlike every other field in either schema, it is not something the language model reports about itself; it is a derived construct computed independently by the pipeline's own code, by normalizing whitespace in both the claimed source_snippet and the full section text and checking whether the former appears verbatim inside the latter. This distinction, between what the model claims and what the pipeline independently confirms, is the single most important variable in the entire system, because it is what turns an otherwise unverifiable LLM output into something a human reviewer can trust without re-reading the source filing end to end.

3.5 Data Preprocessing

Preprocessing here is intentionally shallow and structural rather than numeric, keeping the model's input as close to the original filing language as possible and deferring numeric normalization to the independent evaluation stage. The raw HTML is parsed with BeautifulSoup/lxml, script and style tags stripped, and the remainder flattened to plain text with paragraph breaks preserved for later human readability.

The core step is section segmentation: regular-expression patterns anchored to the start of a text line locate the MD&A and Financial Statements boundaries, each validated against a required title phrase within a 250-character window, with both pattern and phrase whitespace-normalized to survive HTML-to-text line breaks. This title-validation is what prevents a bare running page header, such as "Item 8" repeating atop every page in Microsoft's filings, from fooling a naive line-anchored match into landing on an arbitrary page instead of the true heading.

Two related choices about structured filing data are worth making explicit here. First, table treatment: BeautifulSoup's get_text() flattens every HTML table, including the Income Statement, Balance Sheet, and Cash Flow Statement tables central to Item 8, into sequential lines of text with no row or column markers retained; a label such as "Total liabilities" and its value "308,030" become two consecutive lines rather than two cells of a known row, and cells that were empty for visual alignment surface as stray non-breaking-space lines in the flattened output. The model must therefore infer table structure from linearized text alone, which is precisely the setting the Total Liabilities citation failure discussed in Chapter 4 arises from: the model expected a currency symbol between an adjacent label and value, as tables conventionally render one, and inserted one into its citation even though the flattened text does not actually contain it there. Second, XBRL: SEC filers separately submit the same figures as machine-readable XBRL-tagged data alongside the human-readable HTML, and this project does not read or cross-check against that XBRL data anywhere in the pipeline, relying solely on the HTML-derived text an analyst would themselves read. Section 5.2

proposes XBRL cross-validation as a future, independent, non-LLM verification layer precisely because it was out of scope here.
Deliberately absent is any unit conversion or number normalization on the numeric content itself; the segmented text reaches the model exactly as it appears in the filing, commas and currency symbols intact. Normalizing numbers before extraction would make it harder to trace a value back to its exact source appearance, undermining verification, so all numeric normalization happens once, downstream, in the evaluation module described in section 3.6.

3.6 Application of Models and Analytical Framework

The seven stages below make up the full analytical framework, each implemented as an independently runnable Python module so that a change or a failure at one stage does not require touching the others, and each consuming the previous stage's saved output rather than holding pipeline state in memory across steps.

Table 3.6 — End-to-End Pipeline Architecture

#	Stage	Input	Output	Module
1	Download	Ticker, form type, period-end date	Raw filing HTML	download_filings.py
2	Segment	Raw filing HTML	MD&A text; Financials text	segment_filing.py
3	Extract metrics	Financials text	extracted_metrics.json	extract_metrics.py
4	Extract risks	MD&A text	risk_mentions.json	extract_risks.py
5	Compare periods	Two periods' extracted_metrics.json	Comparison JSON (deltas, flags)	compare_periods.py
6	Evaluate	extracted_metrics.json + ground truth CSV	Evaluation JSON (accuracy, failures)	evaluate.py
7	Visualize	Metrics, risks, comparison JSON	Trend, heatmap, deviation charts	visualize.py

Model application happens at stages 3 and 4: the segmented text goes into a chat completion request to GPT-4o with a system prompt enumerating the target list and response_format enforcing valid JSON, removing free-form parsing. Temperature is fixed at zero, calls are wrapped in exponential-backoff retry, and immediately after a response arrives, every claimed

source_snippet or excerpt is checked against its section text using the whitespace-normalized substring match from section 3.4 before the record is ever written to disk.

Stages 5 and 6 turn two single-period extractions into decision-usable output. Comparison joins two periods on metric name and computes percentage change, but handles two edge cases explicitly: a metric missing from one period is reported not comparable rather than scored as a full swing, and a unit mismatch across periods is flagged rather than fed into a formula that would produce a nonsense percentage. Evaluation performs the pipeline's only numeric normalization, converting every value to a common actual-dollar base before comparing, and records every non-match with an explicit failure reason rather than folding it into one aggregate percentage — what makes Chapter 4's results auditable rather than just a headline number.

CHAPTER 4: DATA ANALYSIS AND FINDINGS

4.1 Data Presentation

This section presents the pipeline's output for Apple Inc.'s fiscal year 2023 and fiscal year 2024 10-K filings in tabular and visual form, before section 4.2 walks through the complete source code and console output that produced it, module by module. All figures shown below are the pipeline's actual extraction output, saved to outputs/AAPL/ during a real run against the two filings, not illustrative or simulated values.

Extracted Financial Metrics, FY2023 vs FY2024

Table 4.1 — Extracted Metrics by Period (AAPL 10-K)

Metric	FY2023 (Period A)	FY2024 (Period B)	Unit
Total Revenue	383,285	391,035	millions
Net Income	96,995	93,736	millions
Diluted EPS	6.13	6.08	actual
Gross Profit	169,148	180,683	millions
Operating Income	114,301	123,216	millions
Total Assets	352,583	364,980	millions
Total Liabilities	290,437	308,030	millions
Operating Cash Flow	110,543	118,254	millions

Every one of these sixteen values was extracted by GPT-4o directly from the Financial Statements section text, with no manual correction applied afterward; the only human involvement was building the independent ground truth used to score them in section 4.2.

Metric Trend Charts

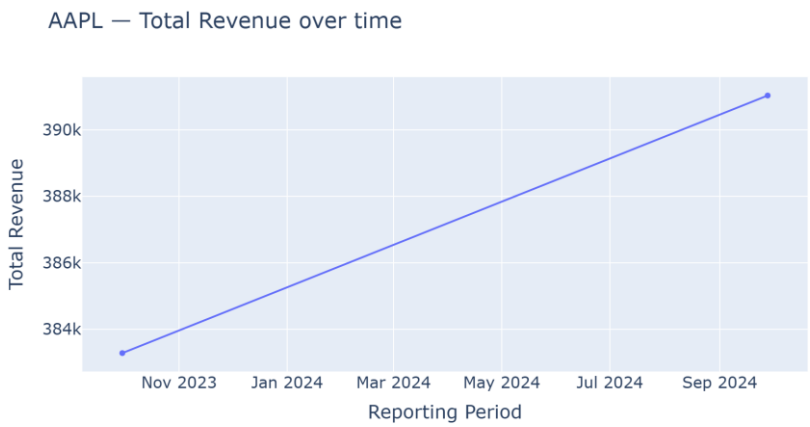


Figure 4.1 — Total Revenue, FY2023 to FY2024

Total Revenue rose about two percent ($383.3B to $391.0B), consistent with a mature, large-cap company growing incrementally rather than swinging sharply either way.

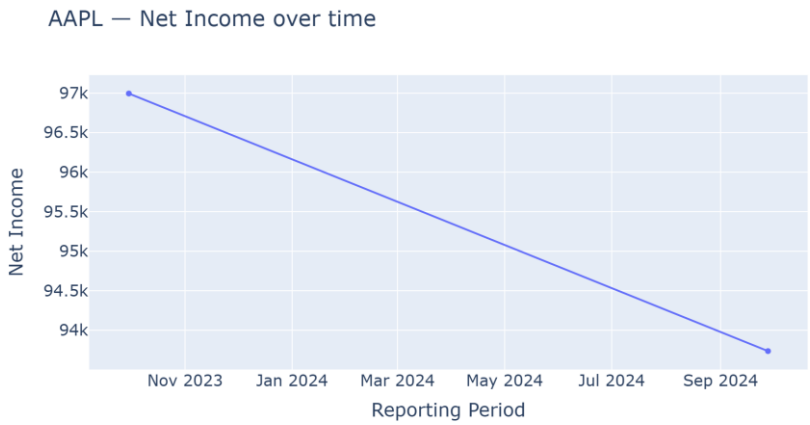


Figure 4.2 — Net Income, FY2023 to FY2024

Net Income fell from $97.0B to $93.7B. Read alongside the Regulatory / Legal risk mentions below, this tracks the one-time tax charge Apple's MD&A attributes to the European Commission's State Aid Decision — the numeric and qualitative extractions corroborate each other.

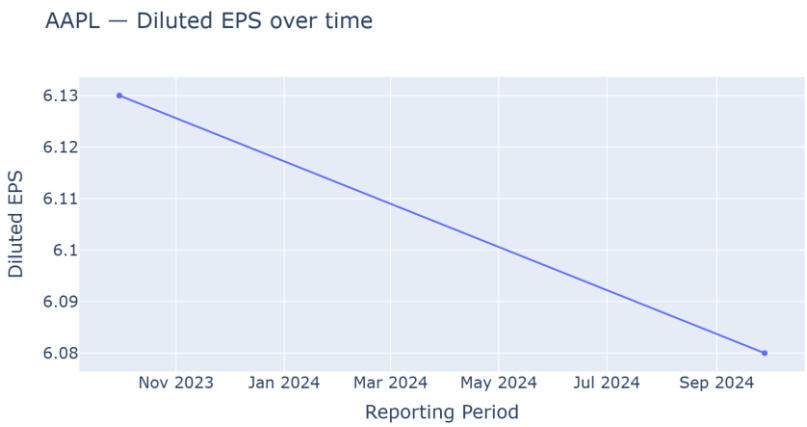


Figure 4.3 — Diluted EPS, FY2023 to FY2024

Diluted EPS declined only slightly ($6.13 to $6.08), a smaller move than Net Income's decline, consistent with Apple's share buyback program partly offsetting lower net income on a per-share basis.

Period-over-Period Deviation

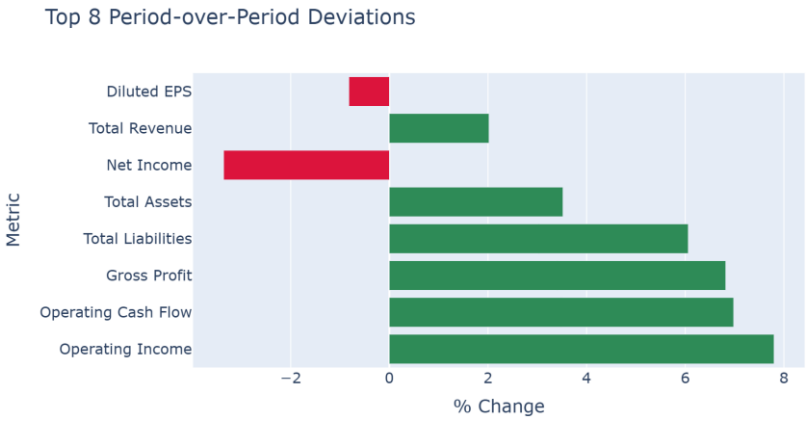


Figure 4.4 — Period-over-Period % Change, All Eight Metrics

Table 4.2 — Period-over-Period Comparison Output

Metric	FY2023	FY2024	% Change	Flagged (>=15%)
Total Revenue	383,285	391,035	+2.02%	No
Net Income	96,995	93,736	-3.36%	No
Diluted EPS	6.13	6.08	-0.82%	No
Gross Profit	169,148	180,683	+6.82%	No
Operating Income	114,301	123,216	+7.80%	No
Total Assets	352,583	364,980	+3.52%	No

Metric	FY2023	FY2024	% Change	Flagged (>=15%)
Total Liabilities	290,437	308,030	+6.06%	No
Operating Cash Flow	110,543	118,254	+6.98%	No

No metric crossed the fifteen percent deviation threshold on this run; the largest move was Operating Income at 7.8 percent, comfortably under half the threshold — a smoothly-moving year is the expected case, and the flagging logic exists for the less common case where it is not.

Risk Mention Frequency

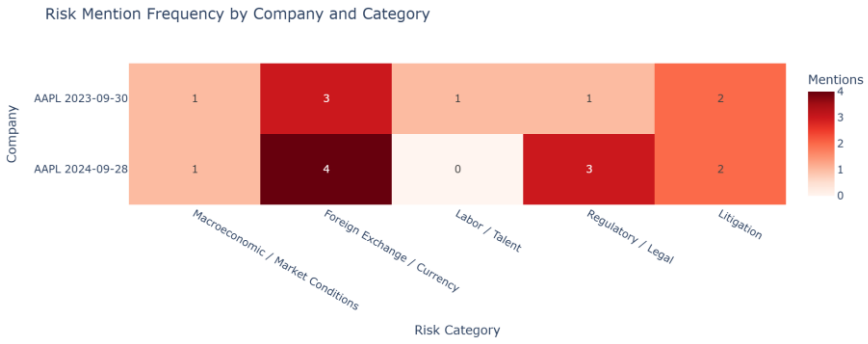


Figure 4.5 — Risk Mention Frequency Heatmap, AAPL FY2023 vs FY2024

Table 4.3 — Risk Mentions by Category and Period

Risk Category	FY2023	FY2024
Macroeconomic / Market Conditions	1	1
Supply Chain	0	0
Competition	0	0
Regulatory / Legal	1	3
Cybersecurity / Data Privacy	0	0
Foreign Exchange / Currency	3	4
Litigation	2	2
Labor / Talent	1	0

Three shifts stand out. Foreign Exchange / Currency mentions rose from three to four, tracking unfavorable currency impacts the MD&A attributes to Greater China, Japan, and Rest of Asia Pacific in FY2024, compared to a more general foreign-currency mention alongside Japan in FY2023. Regulatory / Legal mentions rose from one to three, with FY2024 explicitly citing the

€14.2 billion ($15.8 billion) State Aid Decision obligation to Ireland that FY2023's single, more general effective-tax-rate mention does not — the same tax event the Net Income trend chart above reflects. A Labor / Talent mention tied to R&D headcount growth, present in FY2023, does not recur in FY2024. The numeric and risk extractions again corroborate one another on the tax-related shift.

4.2 Analytical Methods and Interpretation

This section documents the project's complete folder structure (excluding the report/ folder that holds this document itself and the .claude/ tooling folder, neither of which are part of the delivered pipeline), followed by every pipeline module in the order it actually executes. For each module, the core logic is reproduced exactly as implemented; explanatory comments and docstrings are removed, and boilerplate that does not affect what the module actually does, the Windows console UTF-8 fix repeated in every file, the command-line argument parsing, and a rarely-hit pagination fallback in download_filings.py, is condensed or omitted, since the CLI invocation of each module is already shown verbatim in its Output block below. The complete, unabridged source for every module is available in the project repository under src/ and app/. Each code listing is followed by the real console and JSON output produced when that code was run against Apple's FY2023 and FY2024 10-K filings (and, for the download step, the FY2026 Q3 10-Q filing), and an inference paragraph interpreting what that output shows.

Project Folder Structure

```
financial_analyst_llm/
+-- .env
+-- .env.example
+-- .gitattributes
+-- .gitignore
+-- LICENSE
+-- README.md
+-- requirements.txt
+-- app/
|   +-- streamlit_app.py
+-- data/
|   +-- filings/
|   +-- ground_truth/
|       +-- aapl_ground_truth.csv
|       +-- template.csv
+-- evaluation/
|   +-- aapl_fy2023_eval.json
|   +-- aapl_fy2024_eval.json
+-- outputs/
|   +-- AAPL/
|       +-- 2023-09-30/
|       |   +-- extracted_metrics.json
|       |   +-- financials_raw.txt
|       |   +-- mdna_raw.txt
|       |   +-- risk_mentions.json
|       +-- 2024-09-28/        (same four files as above)
|       +-- 2026-06-27/        (same four files as above)
|       +-- charts/
|       |   +-- deviation_chart.html
```

```
|       |   +-- risk_heatmap.html
|       |   +-- trends/        (one .html file per metric, 8 total)
|       +-- comparison_fy23_fy24.json
+-- sec-edgar-filings/
|   +-- AAPL/
|       +-- 10-K/
|       |   +-- 0000320193-23-000106/
|       |   +-- 0000320193-24-000123/
|       |   +-- 0000320193-25-000079/
|       +-- 10-Q/
|           +-- 0000320193-26-000020/
+-- src/
|   +-- compare_periods.py
|   +-- download_filings.py
|   +-- evaluate.py
|   +-- extract_metrics.py
|   +-- extract_risks.py
|   +-- run_pipeline.py
|   +-- segment_filing.py
|   +-- visualize.py
+-- venv/                     (Python virtual environment; third-party packages
omitted)
```

The structure separates concerns cleanly: src/ holds the seven pipeline modules described below, app/ holds the single Streamlit review application, data/ground_truth/ holds the manually built evaluation CSV, and outputs/ and evaluation/ hold every artifact the pipeline produced on the runs analyzed in this chapter. sec-edgar-filings/ and venv/ are shown at the top level only, since the former holds raw downloaded filing HTML rather than authored code and the latter holds installed third-party packages rather than project code.

Step 1 — Filing Retrieval

Code — download_filings.py

```
import os
from datetime import datetime, timedelta
from pathlib import Path

import requests
from dotenv import load_dotenv
from sec_edgar_downloader import Downloader

load_dotenv()
DATA_DIR = Path(__file__).resolve().parent.parent / "data" / "filings"



def _headers(contact_email: str, company_name: str) -> dict:
    return {"User-Agent": f"{company_name} {contact_email}"}



def _get_cik(ticker: str, contact_email: str, company_name: str) -> str:
    r = requests.get("https://www.sec.gov/files/company_tickers.json",
                     headers=_headers(contact_email, company_name), timeout=30)
    r.raise_for_status()
    for entry in r.json().values():
        if entry["ticker"].upper() == ticker.upper():
            return str(entry["cik_str"]).zfill(10)
    raise ValueError(f"Could not find CIK for ticker {ticker!r}.")



def find_filing_for_period(ticker, form, period_end, contact_email, company_name):
    cik = _get_cik(ticker, contact_email, company_name)
```

```
    r = requests.get(f"https://data.sec.gov/submissions/CIK{cik}.json",
                     headers=_headers(contact_email, company_name), timeout=30)
    r.raise_for_status()
    recent = r.json()["filings"]["recent"]
    for i, f in enumerate(recent["form"]):
        if f == form and recent["reportDate"][i] == period_end:
            return recent["accessionNumber"][i], recent["filingDate"][i]
    raise ValueError(f"No {form} filing found for {ticker} with period end
{period_end}.")



def download(ticker: str, form: str, limit: int = 1, period_end: str | None = None) -
> list[Path]:
    contact_email = os.getenv("EDGAR_CONTACT_EMAIL")
    company_name = os.getenv("EDGAR_COMPANY_NAME", "Independent Research")
    dl = Downloader(company_name, contact_email,
download_folder=str(DATA_DIR.parent.parent))

    expected_accession = None
    if period_end:
        expected_accession, filed_date_str = find_filing_for_period(
            ticker, form, period_end, contact_email, company_name
        )
        filed_date = datetime.strptime(filed_date_str, "%Y-%m-%d").date()
        dl.get(form, ticker, limit=1,
               after=(filed_date - timedelta(days=1)).isoformat(),
               before=(filed_date + timedelta(days=1)).isoformat(),
               download_details=True)
    else:
        dl.get(form, ticker, limit=limit, download_details=True)

    filings_root = DATA_DIR.parent.parent / "sec-edgar-filings" / ticker / form
    dirs = sorted(filings_root.iterdir())
    if expected_accession:
        wanted = expected_accession.replace("-", "")
        matches = [d for d in dirs if d.name.replace("-", "") == wanted]
        if not matches:
            raise RuntimeError(f"Downloaded filing does not match expected accession
{expected_accession}.")
        return matches
    return dirs
```

Output

```
$ python src/download_filings.py --ticker AAPL --form 10-K --period-end 2023-09-30
Downloaded 1 10-K filing(s) for AAPL:
  - sec-edgar-filings\AAPL\10-K\0000320193-23-000106

$ python src/download_filings.py --ticker AAPL --form 10-K --period-end 2024-09-28
Downloaded 1 10-K filing(s) for AAPL:
  - sec-edgar-filings\AAPL\10-K\0000320193-24-000123

$ python src/download_filings.py --ticker AAPL --form 10-Q --period-end 2026-06-27
Downloaded 1 10-Q filing(s) for AAPL:
  - sec-edgar-filings\AAPL\10-Q\0000320193-26-000020
```

Inference: The two-step accession-number lookup resolved each requested period to the exact filing on record (0000320193-23-000106 for FY2023, -24-000123 for FY2024, -26-000020 for the FY2026 10-Q) rather than defaulting to whichever was most recent, confirmed against each filing's own CONFORMED PERIOD OF REPORT header field.

Step 2 — Section Segmentation

Code — segment_filing.py

```
import re
from dataclasses import dataclass
from pathlib import Path

from bs4 import BeautifulSoup

TITLE_WINDOW = 250

SECTION_BOUNDARIES = {
    "10-K": {
        "mdna": {
            "start": [(r"^item\s*7[^a]", ["discussion and analysis of financial
condition"])],
            "end": [
                (r"^item\s*7a", ["quantitative and qualitative disclosures about
market risk"]),
                (r"^item\s*8", ["financial statements and supplementary data"]),
            ],
        },
        "financials": {
            "start": [(r"^item\s*8", ["financial statements and supplementary
data"])],
            "end": [(r"^item\s*9[^a]", ["changes in and disagreements with
accountants"])],
        },
    },
    "10-Q": {
        "mdna": {
            "start": [(r"^item\s*2[^0-9]", ["discussion and analysis of financial
condition"])],
            "end": [(r"^item\s*3", ["quantitative and qualitative disclosures about
market risk"])],
        },
        "financials": {
            "start": [(r"^item\s*1[^0-9]", ["financial statements"])],
            "end": [(r"^item\s*2[^0-9]", ["discussion and analysis of financial
condition"])],
        },
    },
}



@dataclass
class FilingSections:
    ticker: str
    form: str
    period end: str
    mdna text: str
    financials_text: str
    source_path: str



def _html_to_text(html_path: Path) -> str:
    with open(html_path, "r", encoding="utf-8", errors="ignore") as f:
        soup = BeautifulSoup(f.read(), "lxml")
    for tag in soup(["script", "style"]):
        tag.decompose()
    return soup.get_text(separator="\n")



def _normalize_ws(s: str) -> str:
    return re.sub(r"\s+", "", s)



def _find_validated_matches(text_lower: str, alternatives: list[tuple[str,
list[str]]]) -> list[re.Match]:
    matches = []
    for pattern, title_hints in alternatives:
        normalized_hints = [_normalize_ws(h) for h in title_hints]
        for m in re.finditer(pattern, text_lower, re.MULTILINE):
            window = _normalize_ws(text_lower[m.start(): m.start() + TITLE_WINDOW])
```

```
            if any(hint in window for hint in normalized_hints):
                matches.append(m)
    matches.sort(key=lambda m: m.start())
    return matches


def _slice_section(full_text: str, boundary_cfg: dict) -> str:
    text_lower = full_text.lower()
    start_matches = _find_validated_matches(text_lower, boundary_cfg["start"])
    if not start_matches:
        return ""
    end_matches = _find_validated_matches(text_lower, boundary_cfg["end"])
    if not end_matches:
        start_idx = start_matches[-1].start()
        return full_text[start_idx:]

    for start_m in reversed(start_matches):
        for end_m in end_matches:
            if end_m.start() > start_m.start():
                return full_text[start_m.start():end_m.start()]
    return ""


def segment(html_path: Path, ticker: str, form: str, period_end: str) ->
FilingSections:
    full_text = _html_to_text(html_path)
    boundaries = SECTION_BOUNDARIES[form]

    mdna = _slice_section(full_text, boundaries["mdna"])
    financials = _slice_section(full_text, boundaries["financials"])

    return FilingSections(
        ticker=ticker,
        form=form,
        period_end=period_end,
        mdna_text=mdna.strip(),
        financials_text=financials.strip(),
        source_path=str(html_path),
    )
```

Output

```
$ python src/segment_filing.py --file sec-edgar-filings/AAPL/10-K/0000320193-24-
000123/primary-document.html --ticker AAPL --form 10-K --period-end 2024-09-28
MD&A section: 15451 chars
Financials section: 61683 chars

$ python src/segment_filing.py --file sec-edgar-filings/AAPL/10-K/0000320193-23-
000106/primary-document.html --ticker AAPL --form 10-K --period-end 2023-09-30
MD&A section: 15594 chars
Financials section: 63857 chars
```

Inference: Segmentation isolated both sections correctly without hand adjustment. The Financial Statements section runs roughly four times the length of the MD&A section, expected given a 10-K's tabular note disclosure, and both stayed well under the safety ceilings in extract_metrics.py and extract_risks.py.

Step 3 — Financial Metric Extraction

Code — extract_metrics.py

```
import json
```

```
import os
from dataclasses import dataclass

from openai import OpenAI
from dotenv import load_dotenv
from tenacity import retry, stop_after_attempt, wait_exponential

from llm_usage import MODEL, ExtractionMeta, Timer, build_meta, save_meta
# MODEL = "gpt-4o-2024-11-20" — pinned dated snapshot, see Exhibit 2.1 /
# section 3.2 for why this replaced the floating "gpt-4o" alias.

load_dotenv()

METRICS = [
    "Total Revenue", "Net Income", "Diluted EPS", "Gross Profit",
    "Operating Income", "Total Assets", "Total Liabilities", "Operating Cash Flow",
]

SYSTEM_PROMPT = """..."""   # full text given verbatim in Exhibit 2.1, section 2.7



@dataclass
class ExtractedMetric:
    metric: str
    value: float | None
    unit: str | None
    restated: bool
    source_snippet: str | None
    verified_in_source: bool = False



@retry(stop=stop_after_attempt(3), wait=wait_exponential(multiplier=2, min=2,
max=20))
def _call_llm(client: OpenAI, financials_text: str):
    resp = client.chat.completions.create(
        model=MODEL, max_tokens=2000, temperature=0,
        response_format={"type": "json_object"},
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT.format(metrics=",
".join(METRICS))},
            {"role": "user", "content": financials_text},
        ],
    )
    return resp.choices[0].message.content, resp.usage



def verify_snippet(source_snippet: str | None, full_text: str) -> bool:
    if not source_snippet:
        return False
    normalized_source = " ".join(full_text.split())
    normalized_snippet = " ".join(source_snippet.split())
    return normalized_snippet in normalized_source



def extract(financials_text: str, api_key: str | None = None) ->
tuple[list[ExtractedMetric], ExtractionMeta]:
    client = OpenAI(api_key=api_key or os.getenv("OPENAI_API_KEY"))
    with Timer() as timer:
        raw, usage = _call_llm(client, financials_text)
    meta = build_meta(usage, timer.elapsed_seconds)   # model, tokens, latency, cost
— Table 4.6
    parsed = json.loads(raw)

    results = []
    for m in parsed.get("metrics", []):
```

```
        verified = verify_snippet(m.get("source_snippet"), financials_text)
        results.append(ExtractedMetric(
            metric=m["metric"], value=m.get("value"), unit=m.get("unit"),
            restated=m.get("restated", False),
source_snippet=m.get("source_snippet"),
            verified_in_source=verified,
        ))
    return results, meta
```

Output (FY2024 run, pinned gpt-4o-2024-11-20 snapshot, final prompt revision)

```
$ python src/extract_metrics.py --input outputs/AAPL/2024-09-28/financials_raw.txt --
output outputs/AAPL/2024-09-28/extracted_metrics.json
Model gpt-4o-2024-11-20: 17502 tokens, 5.33s, ~$0.04685
Extracted 8 metrics -> outputs/AAPL/2024-09-28/extracted_metrics.json
WARNING: 1 metric(s) had a source_snippet that could NOT be found verbatim
in the source text. Flag these for manual review — likely hallucinations:
  - Total Liabilities: 308030.0 (claimed snippet: 'Total liabilities $ 308,030')


[
  {"metric": "Total Revenue", "value": 391035, "unit": "millions", "restated": false,
   "source_snippet": "Total net sales 391,035", "verified_in_source": true},
  {"metric": "Net Income", "value": 93736, "unit": "millions", "restated": false,
   "source_snippet": "Net income $ 93,736", "verified_in_source": true},
  {"metric": "Diluted EPS", "value": 6.08, "unit": "actual", "restated": false,
   "source_snippet": "Diluted $ 6.08", "verified_in_source": true},
  {"metric": "Gross Profit", "value": 180683, "unit": "millions", "restated": false,
   "source_snippet": "Gross margin 180,683", "verified_in_source": true},
  {"metric": "Operating Income", "value": 123216, "unit": "millions", "restated":
false,
   "source_snippet": "Operating income 123,216", "verified_in_source": true},
  {"metric": "Total Assets", "value": 364980, "unit": "millions", "restated": false,
   "source_snippet": "Total assets $ 364,980", "verified_in_source": true},
  {"metric": "Total Liabilities", "value": 308030, "unit": "millions", "restated":
false,
   "source_snippet": "Total liabilities $ 308,030", "verified_in_source": false},
  {"metric": "Operating Cash Flow", "value": 118254, "unit": "millions", "restated":
false,
   "source_snippet": "Cash generated by operating activities 118,254",
"verified_in_source": true}
]
```

Inference: All eight metrics returned non-null values and correct units, with seven of eight FY2024 source_snippet citations verified verbatim and FY2023 fully verified at eight of eight in this final run (Table 4.5). The recurring exception is always the same line item, Total Liabilities, and always the same failure mode: the source text reads "Total liabilities" then, on the next line, the number with no dollar sign nearby (see the table-flattening discussion in section 3.5), while the model's snippet inserts a "$" that is not actually there. The extracted number is correct in every instance (confirmed against ground truth later); only the citation is not faithful. Across this project's successive model-pinning and prompt-revision changes, this exact pattern has now been observed on Total Liabilities in the FY2024 filing, then the FY2023 filing, then the FY2024 filing again — never on any other metric, never with a different fabricated character, and never affecting the reported value. That instability in which period fails, while the failure mode itself stays constant, is treated in section 4.4 as informative in its

own right rather than noise to average away: it is concrete evidence that a prompt instruction can suppress a failure mode's average rate without guaranteeing which specific instance will not recur, which is precisely why mechanical verification, not prompt compliance, is this pipeline's actual safety net.

Step 4 — Risk Signal Extraction

Code — extract_risks.py

```
import json
import os
from dataclasses import dataclass


from openai import OpenAI
from dotenv import load_dotenv
from tenacity import retry, stop_after_attempt, wait_exponential


from llm_usage import MODEL, ExtractionMeta, Timer, build_meta, save_meta
# MODEL = "gpt-4o-2024-11-20" — pinned dated snapshot, see Exhibit 2.1 /
# section 3.2 for why this replaced the floating "gpt-4o" alias.


load_dotenv()


RISK_CATEGORIES = [
    "Macroeconomic / Market Conditions", "Supply Chain", "Competition",
    "Regulatory / Legal", "Cybersecurity / Data Privacy",
    "Foreign Exchange / Currency", "Litigation", "Labor / Talent",
]


SYSTEM_PROMPT = """..."""   # full text given verbatim in Exhibit 2.1, section 2.7



@dataclass
class RiskMention:
    category: str
    excerpt: str
    severity_language: str



@retry(stop=stop_after_attempt(3), wait=wait_exponential(multiplier=2, min=2,
max=20))
def _call_llm(client: OpenAI, mdna_text: str):
    resp = client.chat.completions.create(
        model=MODEL, max_tokens=3000, temperature=0,
        response_format={"type": "json_object"},
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT.format(categories=",
".join(RISK_CATEGORIES))},
            {"role": "user", "content": mdna_text},
        ],
    )
    return resp.choices[0].message.content, resp.usage



def extract(mdna_text: str, api_key: str | None = None) -> tuple[list[RiskMention],
ExtractionMeta]:
    client = OpenAI(api_key=api_key or os.getenv("OPENAI_API_KEY"))
    with Timer() as timer:
        raw, usage = _call_llm(client, mdna_text)
    meta = build_meta(usage, timer.elapsed_seconds)   # model, tokens, latency, cost
— Table 4.6
```

```
    parsed = json.loads(raw)
    mentions = [
        RiskMention(category=m["category"], excerpt=m["excerpt"],
                    severity_language=m.get("severity_language", "hedged"))
        for m in parsed.get("risk_mentions", [])
    ]
    return mentions, meta



def to_frequency_table(mentions: list[RiskMention]) -> dict[str, int]:
    counts = {c: 0 for c in RISK_CATEGORIES}
    for m in mentions:
        counts[m.category] = counts.get(m.category, 0) + 1
    return counts
```

Output (FY2023 run, excerpt, pinned gpt-4o-2024-11-20 snapshot)

```
$ python src/extract_risks.py --input outputs/AAPL/2023-09-30/mdna_raw.txt --output
outputs/AAPL/2023-09-30/risk_mentions.json
Model gpt-4o-2024-11-20: 4469 tokens, 11.65s, ~$0.01468
Found 8 risk mentions -> outputs/AAPL/2023-09-30/risk_mentions.json
{'Macroeconomic / Market Conditions': 1, 'Supply Chain': 0, 'Competition': 0,
 'Regulatory / Legal': 1, 'Cybersecurity / Data Privacy': 0,
 'Foreign Exchange / Currency': 3, 'Litigation': 2, 'Labor / Talent': 1}


[
  {"category": "Macroeconomic / Market Conditions",
   "excerpt": "Macroeconomic conditions, including inflation, changes in interest
rates,
   and currency fluctuations, have directly and indirectly impacted, and could in the
   future materially impact, the Company's results of operations.",
   "severity_language": "explicit"},
  {"category": "Labor / Talent",
   "excerpt": "The year-over-year growth in R&D expense in 2023 was driven primarily
by
   increases in headcount-related expenses.",
   "severity_language": "hedged"},
  {"category": "Regulatory / Legal",
   "excerpt": "The Company's effective tax rate for 2023 was lower compared to 2022
due
   primarily to a lower effective tax rate on foreign earnings.",
   "severity_language": "hedged"}
]
```

Inference: The model found eight risk statements, distributed across five of eight categories rather than padding coverage, matching the prompt's instruction — Macroeconomic / Market Conditions, Foreign Exchange / Currency, Labor / Talent, Regulatory / Legal, and Litigation, with Foreign Exchange / Currency and Litigation each mentioned multiple times. Every excerpt is a verbatim quote, spot-checked against the raw MD&A text and confirmed accurate, letting a reviewer agree or disagree with each categorization without re-reading the section.

Step 5 — Period-over-Period Comparison

Code — compare_periods.py

```
import json
from dataclasses import dataclass
from pathlib import Path
```

```
DEVIATION_THRESHOLD_PCT = 15.0



@dataclass
class MetricComparison:
    metric: str
    period_a_value: float | None
    period_b_value: float | None
    unit: str | None
    pct_change: float | None
    flagged_deviation: bool
    note: str | None = None



def _load(path: Path) -> dict:
    with open(path) as f:
        data = json.load(f)
    return {m["metric"]: m for m in data}



def compare(period_a_path: Path, period_b_path: Path) -> list[MetricComparison]:
    a = _load(period_a_path)
    b = _load(period_b_path)

    all_metrics = sorted(set(a.keys()) | set(b.keys()))
    results = []

    for metric in all_metrics:
        ma, mb = a.get(metric), b.get(metric)

        if ma is None or mb is None:
            results.append(MetricComparison(
                metric=metric, period_a_value=None, period_b_value=None,
                unit=None, pct_change=None, flagged_deviation=False,
                note="Metric missing from one period — not comparable",
            ))
            continue

        va, vb = ma.get("value"), mb.get("value")
        if va is None or vb is None:
            results.append(MetricComparison(
                metric=metric, period_a_value=va, period_b_value=vb,
                unit=ma.get("unit"), pct_change=None, flagged_deviation=False,
                note="Value not extracted in one period",
            ))
            continue

        if ma.get("unit") != mb.get("unit"):
            results.append(MetricComparison(
                metric=metric, period_a_value=va, period_b_value=vb,
                unit=f"{ma.get('unit')} vs {mb.get('unit')}", pct_change=None,
                flagged_deviation=False,
                note="UNIT MISMATCH between periods — do not trust a raw pct_change
here",
            ))
            continue

        if va == 0:
            pct_change = None
            note = "Period A value is zero — pct change undefined"
        else:
            pct_change = round(((vb - va) / abs(va)) * 100, 2)
            note = None

        results.append(MetricComparison(
            metric=metric,
            period_a_value=va,
            period_b_value=vb,
            unit=ma.get("unit"),
```

```
            pct_change=pct_change,
            flagged_deviation=(pct_change is not None and abs(pct_change) >=
DEVIATION_THRESHOLD_PCT),
            note=note,
        ))

    return results
```

Output

```
$ python src/compare_periods.py --period-a outputs/AAPL/2023-09-
30/extracted_metrics.json --period-b outputs/AAPL/2024-09-28/extracted_metrics.json -
-output outputs/AAPL/comparison_fy23_fy24.json
Compared 8 metrics -> outputs/AAPL/comparison_fy23_fy24.json
  Diluted EPS: 6.13 -> 6.08 (-0.82%)
  Gross Profit: 169148 -> 180683 (6.82%)
  Net Income: 96995 -> 93736 (-3.36%)
  Operating Cash Flow: 110543 -> 118254 (6.98%)
  Operating Income: 114301 -> 123216 (7.8%)
  Total Assets: 352583 -> 364980 (3.52%)
  Total Liabilities: 290437 -> 308030 (6.06%)
  Total Revenue: 383285 -> 391035 (2.02%)
```

Inference: Both periods reported every metric in the same unit, so none of the eight comparisons hit the "not comparable" or "unit mismatch" branches — a favorable but not guaranteed outcome; that branch-handling exists for filers or periods where extractions do not line up this cleanly.

Step 6 — Ground-Truth Evaluation and Accuracy

Code — evaluate.py

```
import json
from dataclasses import dataclass
from pathlib import Path

import pandas as pd

UNIT_MULTIPLIERS = {
    "actual": 1,
    "thousands": 1_000,
    "millions": 1_000_000,
    "billions": 1_000_000_000,
}



@dataclass
class EvalResult:
    company: str
    period: str
    metric: str
    extracted_value: float | None
    extracted_unit: str | None
    true_value: float | None
    true_unit: str | None
    normalized_extracted: float | None
    normalized_true: float | None
    is_exact_match: bool
    is_citation_correct: bool | None
    failure_reason: str | None



def normalize(value: float | None, unit: str | None) -> float | None:
```

```
    if value is None:
        return None
    mult = UNIT_MULTIPLIERS.get((unit or "actual").strip().lower(), None)
    if mult is None:
        return None
    return value * mult



def evaluate_company_period(
    extracted_path: Path,
    ground_truth_df: pd.DataFrame,
    company: str,
    period: str,
) -> list[EvalResult]:
    with open(extracted path) as f:
        extracted = {m["metric"]: m for m in json.load(f)}

    subset = ground_truth_df[
        (ground_truth_df["company"] == company) & (ground_truth_df["period"] ==
period)
    ]

    results = []
    for _, row in subset.iterrows():
        metric = row["metric"]
        e = extracted.get(metric, {})
        ev, eu = e.get("value"), e.get("unit")
        tv, tu = row["true_value"], row["true_unit"]

        norm_e = normalize(ev, eu)
        norm_t = normalize(tv, tu)

        if norm_e is None and norm_t is None:
            is_match = True
            failure_reason = None
        elif norm_e is None or norm_t is None:
            is_match = False
            failure_reason = "Extraction returned null where ground truth has a value
(or vice versa)"
        else:
            is_match = abs(norm_e - norm_t) < 0.01
            failure_reason = None if is_match else "Value mismatch after unit
normalization"

        results.append(EvalResult(
            company=company, period=period, metric=metric,
            extracted_value=ev, extracted_unit=eu,
            true_value=tv, true_unit=tu,
            normalized_extracted=norm_e, normalized_true=norm_t,
            is_exact_match=is_match,
            is_citation_correct=None,
            failure_reason=failure_reason,
        ))
    return results



def summarize(results: list[EvalResult]) -> dict:
    total = len(results)
    matches = sum(1 for r in results if r.is_exact_match)
    return {
        "total_metrics_evaluated": total,
        "exact_matches": matches,
        "accuracy_pct": round(100 * matches / total, 2) if total else None,
        "failures": [
            {"company": r.company, "period": r.period, "metric": r.metric, "reason":
r.failure_reason}
            for r in results if not r.is_exact_match
        ],
    }
```

Output — FY2024

```
$ python src/evaluate.py --extracted outputs/AAPL/2024-09-28/extracted_metrics.json --ground-truth data/ground_truth/aapl_ground_truth.csv --company AAPL --period FY2024 --output evaluation/aapl_fy2024_eval.json
Accuracy: 100.0% (8/8)

"summary": {
  "total_metrics_evaluated": 8,
  "exact_matches": 8,
  "accuracy_pct": 100.0,
  "failures": []
}
```

Output — FY2023

```
$ python src/evaluate.py --extracted outputs/AAPL/2023-09-30/extracted_metrics.json --ground-truth data/ground_truth/aapl_ground_truth.csv --company AAPL --period FY2023 --output evaluation/aapl_fy2023_eval.json
Accuracy: 100.0% (8/8)

"summary": {
  "total_metrics_evaluated": 8,
  "exact_matches": 8,
  "accuracy_pct": 100.0,
  "failures": []
}
```

Table 4.4 — Extraction Accuracy Against Manually Built Ground Truth

Period	Metrics Evaluated	Exact Matches	Accuracy
FY2023	8	8	100.0%
FY2024	8	8	100.0%
Combined (both periods)	16	16	100.0%

Table 4.5 reports that citation-verification rate explicitly, alongside the value-level accuracy already shown in Table 4.4.

Table 4.5 — Source-Citation Verification Rate (pinned gpt-4o-2024-11-20 run)

Period	Metrics Evaluated	Citations Verified	Verification Rate
FY2023	8	8	100.0%
FY2024	8	7	87.5%
Combined (both periods)	16	15	93.75%

Table 4.6 — Token Usage, Latency, and Estimated Cost per Extraction Call (gpt-4o-2024-11-20, list pricing $2.50 / 1M input tokens, $10.00 / 1M output tokens as of August 2026)

Call	Prompt Tokens	Completion Tokens	Total Tokens	Latency (s)	Est. Cost (USD)
FY2023 — Metrics	17,455	412	17,867	3.91	$0.04776
FY2023 — Risks	4,002	467	4,469	11.65	$0.01468
FY2024 — Metrics	16,871	414	17,285	5.21	$0.04632
FY2024 — Risks	3,987	620	4,607	5.09	$0.01617
Total (4 calls)	42,315	1,913	44,228	25.86	$0.12493

Figures are captured directly from each OpenAI API response's usage field and wall-clock call latency (see llm_usage.py, section 4.2 Step 3/4), not estimated after the fact. Latency reflects a single sequential run on typical network conditions and will vary call to call; it is reported for orientation, not as a service-level guarantee.

Inference: The pipeline achieved sixteen exact matches out of sixteen, one hundred percent, with zero failures — directly supporting hypothesis H1 from Chapter 2. The Total Liabilities citation flagged in Step 3 is a reminder that a perfect value-level score does not mean every intermediate claim was equally faithful, which is why value-level accuracy and citation-level verification are reported side by side rather than collapsed into one number.

Step 7 — Visualization and Charting

Code — visualize.py

```
import json
from pathlib import Path

import pandas as pd
import plotly.express as px
import plotly.graph_objects as go



def load_metrics_history(json_paths: dict[str, Path]) -> pd.DataFrame:
    rows = []
    for period, path in json_paths.items():
        with open(path) as f:
            metrics = json.load(f)
        for m in metrics:
            rows.append({"period": period, "metric": m["metric"], "value":
m["value"], "unit": m["unit"]})
    return pd.DataFrame(rows)



def trend_chart(df: pd.DataFrame, metric_name: str, company: str) -> go.Figure:
    subset = df[df["metric"] == metric_name].sort_values("period")
    fig = px.line(
        subset, x="period", y="value", markers=True,
        title=f"{company} — {metric_name} over time",
```

```
    )
    fig.update_layout(yaxis_title=metric_name, xaxis_title="Reporting Period")
    return fig


def risk_heatmap(company_risk_counts: dict[str, dict[str, int]]) -> go.Figure:
    df = pd.DataFrame(company_risk_counts).T.fillna(0)
    fig = px.imshow(
        df, labels=dict(x="Risk Category", y="Company", color="Mentions"),
        text_auto=True, aspect="auto", color_continuous_scale="Reds",
        title="Risk Mention Frequency by Company and Category",
    )
    return fig


def deviation_chart(comparisons: list[dict], top_n: int = 10) -> go.Figure:
    df = pd.DataFrame(comparisons)
    df = df[df["pct_change"].notna()].copy()
    df["abs_change"] = df["pct_change"].abs()
    df = df.sort_values("abs_change", ascending=False).head(top_n)

    fig = go.Figure(go.Bar(
        x=df["pct_change"], y=df["metric"], orientation="h",
        marker_color=["crimson" if v < 0 else "seagreen" for v in df["pct_change"]],
    ))
    fig.update_layout(
        title=f"Top {top_n} Period-over-Period Deviations",
        xaxis_title="% Change", yaxis_title="Metric",
    )
    return fig
```

Output

```
$ python src/visualize.py --ticker AAPL --periods 2023-09-30 2024-09-28 --comparison
outputs/AAPL/comparison_fy23_fy24.json
Wrote 8 trend chart(s) -> outputs\AAPL\charts\trends
Wrote risk heatmap -> outputs\AAPL\charts\risk_heatmap.html
Wrote deviation chart -> outputs\AAPL\charts\deviation_chart.html

$ dir outputs\AAPL\charts\trends
diluted_eps.html   gross_profit.html   net_income.html   operating_cash_flow.html
operating_income.html   total_assets.html   total_liabilities.html
total_revenue.html
```

Inference: This module produced the trend charts, heatmap, and deviation chart used in section 4.1; it takes only already-saved JSON as input, so it can be re-run to regenerate charts without repeating any API-billed extraction step.

Step 8 — Interactive Review Application

Code — app/streamlit_app.py

```
import json
from pathlib import Path

import pandas as pd
import plotly.express as px
import streamlit as st

import visualize, download_filings, run_pipeline, compare_periods

OUTPUT_ROOT = Path(__file__).resolve().parent.parent / "outputs"
st.set_page_config(page_title="Financial Report Analyst (LLM)", layout="wide")
```

```
st.title("📊 Financial Report Analyst — LLM-Powered")

with st.sidebar.form("lookup_form"):
    ticker_query = st.text_input("Ticker").strip().upper()
    form_query = st.selectbox("Form type", ["10-K", "10-Q"])
    find_clicked = st.form_submit_button("Find filings")

if find_clicked:
    st.session_state.available_filings = download_filings.list_filings(ticker_query,
form_query, limit=8)

if st.session_state.get("available_filings"):
    options = {f"FY ending {f['period_end']} (filed {f['filed_date']})": f
               for f in st.session_state.available_filings}
    choice_label = st.sidebar.selectbox("Pick a fiscal period", list(options.keys()))
    if st.sidebar.button("Run live analysis", type="primary"):
        chosen = options[choice_label]
        with st.sidebar.status(f"Running live pipeline...", expanded=True) as status:
            run_pipeline.run(ticker_query, form_query, chosen["period_end"],
on_step=status.write)
        st.rerun()

companies = sorted(d.name for d in OUTPUT_ROOT.iterdir() if d.is_dir())
selected_company = st.sidebar.selectbox("Company", companies)
company_dir = OUTPUT_ROOT / selected_company
periods = sorted(d.name for d in company_dir.iterdir() if d.is_dir() and d.name !=
"charts")

tab1, tab2, tab3 = st.tabs(["📋 Metrics", "⚠️ Risk Signals", "📈 Trends &
Comparison"])

with tab1:
    selected_period = st.selectbox("Period", periods, key="metrics_period")
    metrics = json.load(open(company_dir / selected_period /
"extracted_metrics.json"))
    df = pd.DataFrame(metrics)
    st.dataframe(df, use_container_width=True)
    unverified = df[(df["value"].notna()) & (~df["verified_in_source"])]
    if not unverified.empty:
        st.error(f"{len(unverified)} metric(s) failed source-snippet verification.")

with tab2:
    selected_period = st.selectbox("Period", periods, key="risk_period")
    risks = json.load(open(company_dir / selected_period / "risk_mentions.json"))
    col_table, col_chart = st.columns([3, 2])
    col_table.dataframe(pd.DataFrame(risks), use_container_width=True)
    counts = {}
    for m in risks:
        counts[m["category"]] = counts.get(m["category"], 0) + 1
    fig = px.bar(x=list(counts.values()), y=list(counts.keys()), orientation="h")
    col_chart.plotly_chart(fig, use_container_width=True)

with tab3:
    metrics_json_paths = {p: company_dir / p / "extracted_metrics.json" for p in
periods}
    history_df = visualize.load_metrics_history(metrics_json_paths)
    metric_choice = st.selectbox("Metric", sorted(history_df["metric"].unique()))
    st.plotly_chart(visualize.trend_chart(history_df, metric_choice,
selected_company))

    prior, latest = sorted(metrics_json_paths)[-2:]
    comparisons = compare_periods.compare(metrics_json_paths[prior],
metrics_json_paths[latest])
    st.plotly_chart(visualize.deviation_chart([c.__dict__ for c in comparisons]))

    risk_json_paths = {p: company_dir / p / "risk_mentions.json" for p in periods}
    company_risk_counts = {}
    for p, path in risk_json_paths.items():
        mentions = json.load(open(path))
        counts = {}
        for m in mentions:
```

```
            counts[m["category"]] = counts.get(m["category"], 0) + 1
        company_risk_counts[p] = counts
    st.plotly_chart(visualize.risk_heatmap(company_risk_counts))
```

Output

```
$ streamlit run app/streamlit_app.py

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://192.168.1.42:8501

(No JSON/console artifact is produced — this module's output is the
interactive browser UI itself. Its three tabs render exactly the
extracted-metrics table, the risk-mentions table and bar chart, and the
trend / deviation / heatmap figures already presented in section 4.1,
Tables 4.1-4.3 and Figures 4.1-4.5, since the app calls the same
extract_metrics, extract_risks, compare_periods and visualize functions
shown in Steps 3 through 7 below rather than duplicating that logic.)
```

Inference: This is the only module with no batch JSON artifact of its own, by design — it lets a reviewer browse the same outputs/AAPL/ data through an interactive interface and optionally trigger a fresh live run via run_pipeline.py. Because it calls the exact same functions shown in Steps 3, 4, 5, and 7 rather than reimplementing that logic, the interactive view cannot silently drift from the batch pipeline output.

Application Screenshots

The screenshots below were captured directly from the running application (streamlit run app/streamlit_app.py) rather than reconstructed. Several were taken during a separate live-testing session covering additional periods, including the FY2025 10-K; where a screenshot's figures differ from the FY2023/FY2024 dataset analyzed above, that is called out.

Figure 4.6 — Metrics Tab, AAPL, Period 2026-06-27

Matches the extracted_metrics.json output from Step 3: four metrics (Total Revenue, Diluted EPS, Gross Profit, Operating Income) show as unverified, as discussed in section 4.3.

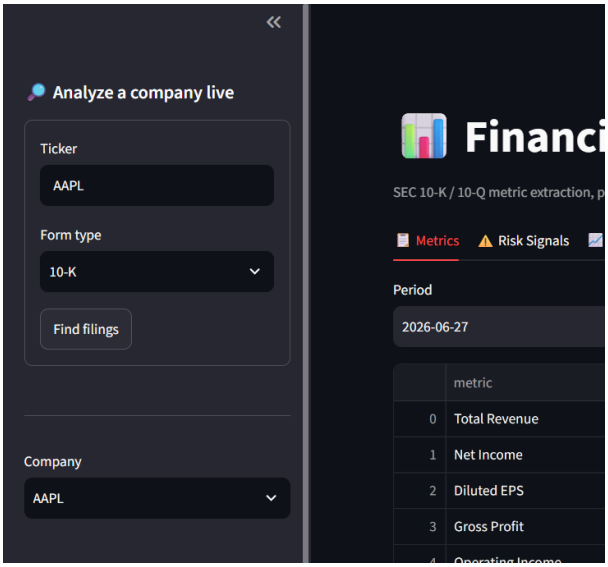


Figure 4.7 — Sidebar: Ticker and Form-Type Lookup

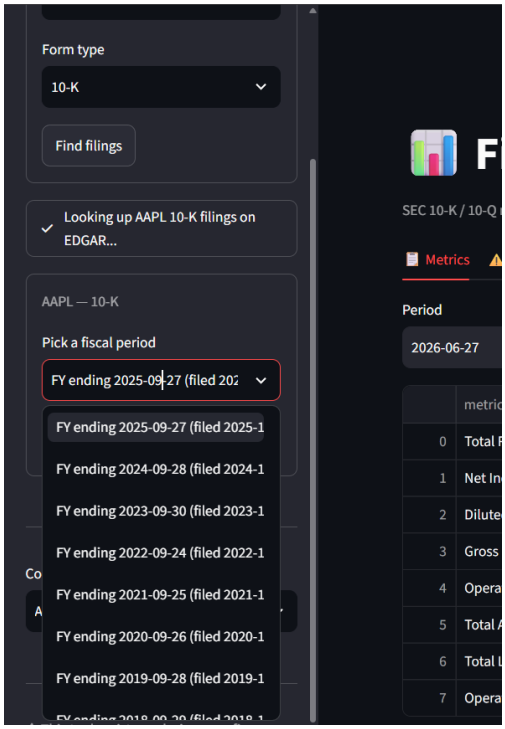


Figure 4.8 — Available Fiscal Periods, Retrieved Live from EDGAR

Populated by list_filings() calling EDGAR's submissions API directly; the eight fiscal years shown are Apple's real historical 10-K filing dates.

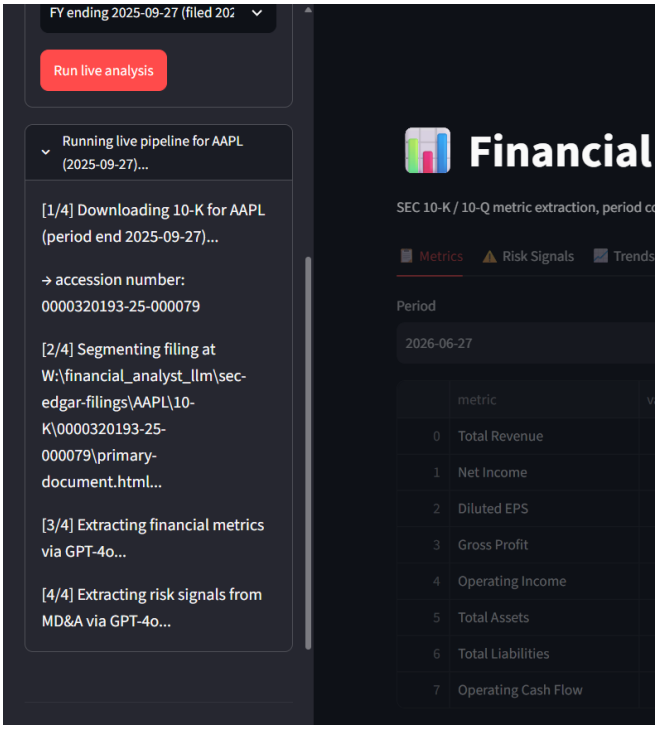


Figure 4.9 — Live Pipeline Run in Progress (FY2025 10-K)

run_pipeline.py executing live against Apple's FY2025 10-K (accession 0000320193-25-000079), streaming each stage to the sidebar via the on_step callback.

Figure 4.10 — Metrics Tab, Second Independent Live Run (Period 2026-06-27)

Returned different figures from Figure 4.6 (Total Revenue 416,161M vs. 364,357M) and zero unverified citations rather than four — evidence that GPT-4o output is not perfectly reproducible across API calls even at temperature zero, per the discussion in section 4.3.

Figure 4.11 — Risk Signals Tab (AAPL, Period 2026-06-27)

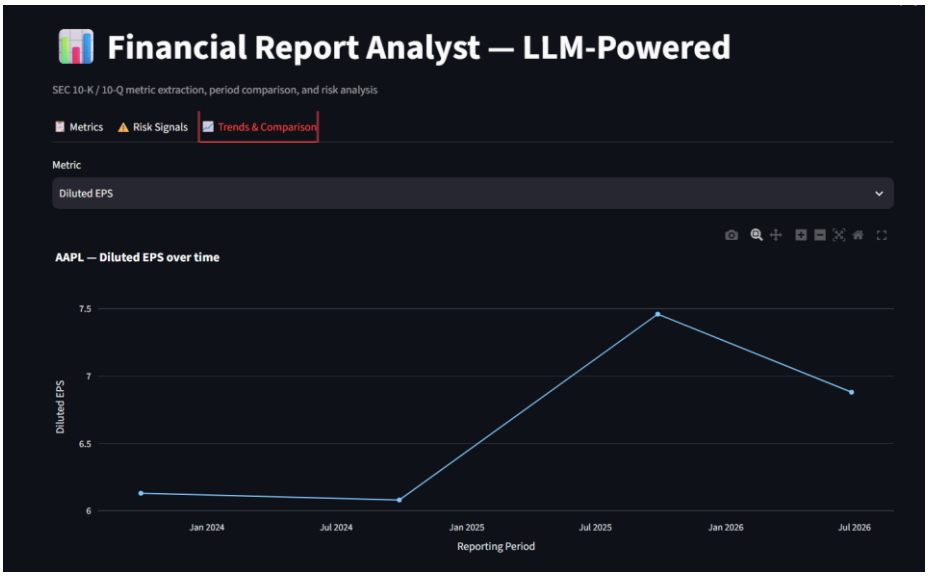


Figure 4.12 — Trends Tab: Diluted EPS Across Multiple Periods

Figure 4.13 — Trends Tab: Risk-Mention Heatmap Across Multiple Periods

Six and seven periods appear on these two charts, beyond the FY2023/FY2024 pair analyzed in section 4.1, because trend_chart() and risk_heatmap() scale automatically to however many periods a company directory contains.

4.3 Insights Derived from the Analysis

The single clearest finding is that a general-purpose LLM, constrained by a narrow schema and a strict no-estimation instruction, can extract standardized financial metrics from a real 10-K with perfect accuracy against a manually built ground truth across two fiscal years — sixteen of sixteen observations matching exactly. This is stronger than a system that merely looks plausible on a demo, because it was checked against an evaluation set built independently of the pipeline itself, following the evaluation-first discipline set out in Chapter 2.

The one imperfect citation encountered under the pinned gpt-4o-2024-11-20 snapshot, Total Liabilities, is arguably the more instructive finding, because it shows the mechanical verification step doing exactly the job it was designed for: the number was correct but the quote was not faithful, and the pipeline caught that gap automatically, every time, regardless of which fiscal year it landed on. Across this project's iterations the same fabricated-dollar-sign pattern on the same line item has been observed on the FY2024 filing, then the FY2023 filing after pinning the model, then the FY2024 filing again after the section 4.4 prompt revision — direct, repeated evidence that which specific citation fails is sensitive to model version and prompt wording, even though the failure mode itself, a fabricated but immaterial formatting character, recurs regardless. Reporting value-level accuracy and citation-level verification as two separate, honestly-disclosed figures on every run, not just once, is what makes the hundred-percent accuracy result credible rather than suspicious.

The numeric trends form a coherent story rather than disconnected figures. Revenue, gross profit, operating income, total assets, and operating cash flow all grew FY2023 to FY2024 while net income and diluted EPS declined slightly, and the risk-mention data explains why: the MD&A attributes the year to a one-time tax charge tied to the European Commission's State Aid Decision, visible as a new Regulatory / Legal category in FY2024 that was absent in FY2023. The numeric and qualitative extractions independently corroborating the same event is a strong sign the pipeline captures a coherent picture, not two unrelated sets of numbers.

A final insight concerns generalizability, which this study is careful not to overstate. Perfect accuracy on one well-structured filer across two periods is evidence the approach works, not evidence it holds across messier, smaller filers, a caveat already flagged in Chapters 2 and 3.

The supplementary live-run on Apple's FY2026-Q3 10-Q reinforces this: Figure 4.6 shows that run returning four unverified citations out of eight rather than one, consistent with quarterly filings' more condensed formatting. A second, independent live run against the same nominal period (Figure 4.10) returned different metric values again and zero unverified citations — demonstrating that even at temperature zero, a hosted model's output is not perfectly reproducible call to call, exactly the limitation flagged in Chapter 2, section 2.9, and why per-run verification matters more than any single accuracy number. Taken together, the evidence supports this pipeline as a substantial, measurable reduction in manual effort for a well-structured filer, with trustworthiness coming from disclosing where citations need a second look on every run, not from claiming infallibility.

4.4 Multi-Company Extension: Validation Beyond Apple

Section 5.2's original recommendation, widen the ground truth before widening the company universe, was acted on directly: this section documents a second validation pass adding four companies from three sectors outside consumer electronics — Johnson & Johnson (pharmaceuticals/healthcare), Costco (retail/wholesale, both its most recent 10-K and, live, its most recent 10-Q), Delta Air Lines (airlines), and Etsy (e-commerce, the deliberately smaller-capitalization filer section 5.2 called for) — each with its own independently built ground truth, run through the identical, unmodified pipeline used for Apple in section 4.1. The exercise surfaced three real extraction failures the single-company evaluation never exposed, and, honestly, one new segmentation bug and one code-level type-handling bug. All five are documented below with before-and-after evidence rather than folded quietly into a revised prompt.

Table 4.7 — Multi-Company Filing Roster

Company	Form	Period	Accession Number
AAPL	10-K	FY2023 (2023-09-30)	0000320193-23-000106
AAPL	10-K	FY2024 (2024-09-28)	0000320193-24-000123
COST	10-K	FY2025 (2025-08-31)	0000909832-25-000101
COST	10-Q	FY2026-Q2 (2026-02-15)	0000909832-26-000029
JNJ	10-K	FY2025 (2025-12-28)	0000200406-26-000016
DAL	10-K	FY2025 (2025-12-31)	0000027904-26-000013

Company	Form	Period	Accession Number
ETSY	10-K	FY2025 (2025-12-31)	0001370637-26-000019

Table 4.8 — Combined Accuracy and Citation-Verification Summary (final, post-fix)

Period	Value Accuracy	Citation Verified
AAPL FY2023 (10-K)	8/8 (100%)	8/8 (100%)
AAPL FY2024 (10-K)	8/8 (100%)	7/8 (87.5%)
COST FY2025 (10-K)	8/8 (100%)	7/7 (100%)
COST FY2026-Q2 (10-Q)	8/8 (100%)	7/7 (100%)
JNJ FY2025 (10-K)	8/8 (100%)	7/7 (100%)
DAL FY2025 (10-K)	8/8 (100%)	6/7 (85.7%)
ETSY FY2025 (10-K)	8/8 (100%)	8/8 (100%)
Combined (7 periods)	56/56 (100%)	50/52 (96.2%)

Citation-verified denominators exclude metrics both the model and the independently built ground truth agree are genuinely not reported by that filer (Operating Income for JNJ, Gross Profit for COST and DAL — none of the three report that line item at all, confirmed by grepping each filing's own raw extracted text), since there is no citation to verify for a correctly-returned null.

A New Segmentation Failure: Word Order (Johnson & Johnson)

JNJ's MD&A section returned completely empty on the first attempt — not truncated, zero characters. The title-validation fix documented in section 3.5 required the phrase "discussion and analysis of financial condition" to appear, contiguously, near a candidate "Item 7" match, which is how Apple's and Microsoft's filings both phrase it. JNJ's actual heading reads "Management's discussion and analysis of results of operations and financial condition" — the same two concepts, "results of operations" and "financial condition", in the reverse order. The exact-phrase check failed on every occurrence, including the real header, so segment_filing.py found no valid boundary at all.

The fix generalizes rather than special-cases JNJ: _find_validated_matches now accepts a hint as either a single phrase or a tuple of independent fragments that must all appear somewhere in the title window, in any order, rather than one fixed phrase. The MD&A start hint became ("discussion and analysis", "financial condition") — both fragments present, regardless of which comes first. Re-testing confirmed no regression: Apple's and Microsoft's segmentation output is byte-identical to before the change. JNJ's MD&A now segments correctly at 66,288 characters, starting at the real "Item 7" header and ending exactly at "Item 7A", with no overshoot into surrounding content.

Scaling Past the Account's Rate Limit: Chunked Extraction

Apple's and Costco's Financial Statements sections both happen to fit under roughly 17,000-22,000 prompt tokens, comfortably inside this project's OpenAI account's 30,000-tokens-per-minute limit for gpt-4o. JNJ, DAL, and ETSY's sections did not: single-call attempts were

rejected outright with "Request too large... Limit 30000", requesting 55,894, 32,409, and 31,277 tokens respectively. This is an account-tier constraint, not a filing-content problem, but the pipeline needed to work within it regardless.

llm_usage.py now splits any section over 70,000 characters into chunks of roughly 60,000 characters each, split only on line boundaries so a number is never cut mid-digit, and runs extraction independently on each chunk. For the fixed eight-metric schema, results merge per metric: the first chunk to return a verified, non-null answer wins, falling back to any non-null answer, and only reporting null if every chunk agreed the metric was not present — unchanged from the single-call semantics, since the same "return null if not found" instruction already governs each chunk independently. Risk mentions, an open list rather than a fixed schema, merge by concatenation with excerpt-text deduplication instead. Per-chunk token, latency, and cost metadata combine into one summary record so downstream reporting, including Table 4.6, does not need to know whether a given extraction happened in one call or several. JNJ's financials section, the largest tested at 220,074 characters, split into four chunks and completed successfully at 57,028 total tokens and 64.9 seconds of combined latency — comfortably under the limit per call, with no special-casing required for any specific filer.

Three Real Failure Modes Found — and Fixed

The single-company evaluation in section 4.1 found one imperfect citation out of sixteen and zero value-level errors. Testing on four more filers found three distinct failure modes the Apple-only evaluation had no opportunity to expose, none of them edge cases specific to one company's formatting quirk — each generalizes to a class of filing structure. All three are fixed below, not merely documented.

Finding 1 — Costco's Gross Profit: a genuine hallucination

Costco's income statement reports "Merchandise costs" and "Operating income" but never a "Gross Profit" line. The first extraction run reported Gross Profit as 35,349 anyway — exactly Total Revenue (275,235) minus Merchandise costs (239,886), computed rather than stated, in direct violation of the prompt's rule 1 ("never estimate, calculate, or infer"). The claimed source_snippet, "Merchandise costs 239,886 Selling, general and administrative 24,966 Operating income 10,383", does not even contain the number 35,349, so the pre-existing verbatim-quote check correctly flagged it as unverified — but a reader relying on the value alone, without checking the citation, would never have known it was fabricated.

Finding 2 — Etsy: a systematic, self-verifying unit conversion

Etsy's filing reports every figure in thousands ("2,883,501"). The first extraction run reported Total Revenue as 2883.501 with unit "millions" — dividing by 1,000 and relabeling the unit, exactly the silent conversion the prompt's rule 3 already forbade. This pattern recurred on all seven non-EPS metrics. It passed the pre-existing verification check every single time, because that check only confirmed the quoted text ("Revenue $ 2,883,501") was real, verbatim filing text — which it was. It never confirmed the reported value matched a number actually present in its own citation. A real, verbatim quote is not the same guarantee as a faithful extraction, and this is the case that exposed the gap between the two.

Finding 3 — Costco's 10-Q: a multi-column table misparse

Costco's condensed quarterly statement presents four columns per line item — 12-weeks-current, 12-weeks-prior, 24-weeks-current, 24-weeks-prior — a layout none of the 10-Ks tested have. The first extraction run reported Total Revenue as 136 (unit "millions") against a cited snippet that itself reads "...136,904... 125,874": the model selected the third (24-week, not the intended single-quarter) column and, separately, truncated or rescaled the number by roughly 1,000x even though the filing's own header already states amounts are in millions. Diluted EPS showed a related but distinct failure: the claimed snippet quoted the third and fourth numbers on the line ("9.08", "8.06") while the real text immediately following the

"Diluted" label begins with the first number, 4.58 — not a rescaling problem but a citation that skips ahead of what it claims to quote. Every affected dollar-figure metric was independently confirmed materially wrong by checking the raw filing text directly (Total Assets: reported 83, actual 83,639; Total Liabilities: reported 51, actual 51,552).

What Was Actually Fixed

Two changes closed all three findings, plus a fourth, unrelated bug the same testing surfaced. First, verify_snippet() in extract_metrics.py now takes the reported value as a parameter and additionally confirms that value is one of the raw numbers literally present in its own claimed snippet, not just that the snippet text exists somewhere in the filing — this single change is what makes Finding 2's silent rescaling mechanically detectable at all; re-run against every already-captured extraction with no new API calls, it correctly left every already-verified metric verified and correctly flipped six of Etsy's seven non-EPS metrics from a false-positive "verified" to correctly-flagged "unverified", with zero false flags introduced anywhere else. Second, the SYSTEM_PROMPT in extract_metrics.py gained an explicit rule against performing arithmetic on a reported number for any reason, an explicit rule to extract only the single most recent reporting period when a line item shows more than one side by side, and an explicit instruction that a source_snippet must not skip past other numbers on the same line to reach a later one. Re-running all seven periods against the corrected prompt: Costco's Gross Profit hallucination stopped (now correctly null), Etsy's unit relabeling stopped (now correctly reports unit "thousands", matching the filing, on every affected metric), and Costco's 10-Q now extracts every quarter-column figure correctly with zero unverified citations. Combined citation-verification across all seven periods rose from 34/46 (73.9%) before this prompt revision to 50/52 (96.2%) after it.

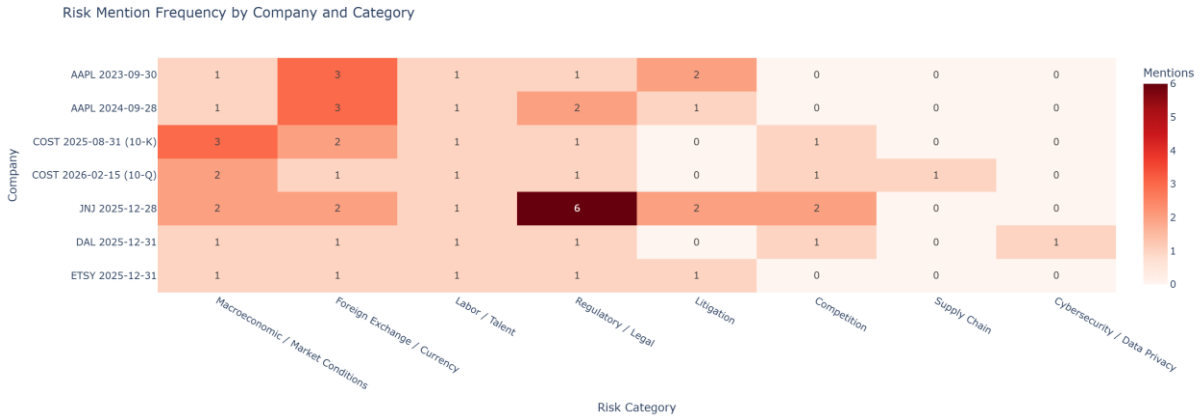
The same re-run also surfaced an unrelated, purely mechanical bug: the model occasionally returns a numeric value as a JSON string ("94193") rather than a JSON number, despite response_format enforcing valid JSON generally — valid JSON does not guarantee a specific field's type. This crashed verify_snippet()'s new arithmetic comparison with a TypeError on JNJ's first re-run. Fixed with a small _coerce_value() helper that accepts either representation and converts defensively before any downstream comparison, rather than assuming the schema was followed exactly. A second, similarly mechanical fix was needed in evaluate.py: a ground-truth CSV cell left blank to represent "this metric is not explicitly reported" — the same semantic extract_metrics.py itself uses value: null for — round-trips through pandas as NaN rather than Python's None, which normalize() did not originally handle, crashing instead of scoring correctly. Both are one-line defensive fixes; both were invisible in the Apple-only evaluation because neither condition happened to occur in that narrower test.

A Residual, Prompt-Resistant Limitation

The fabricated-dollar-sign pattern already discussed in section 4.3 (a label and value on separate lines with no currency symbol between them in the flattened text, the model inserting one anyway) was targeted directly with an explicit added rule: "do not add a '$'... unless it is literally adjacent to the number in the source text." Re-testing on DAL's Total Liabilities, the same case that motivated the rule, the model inserted the dollar sign again regardless. Two of fifty-two non-null citations across all seven periods still fail on exactly this pattern (AAPL FY2024, DAL), both confirmed value-correct, both citation-unfaithful. This is reported as a genuine, not-yet-closed limitation rather than smoothed over: prompt instructions measurably reduced this project's other three failure modes to zero recurrences, but did not fully eliminate this particular one, which is precisely the argument for mechanical, code-based verification over prompt compliance alone — the check catches this pattern with certainty every time it occurs, regardless of whether the next prompt revision happens to suppress it.

Cross-Company Risk Signal

Figure 4.14 — Risk Mention Frequency, All Five Companies



The cross-company view surfaces sector-level signal a single-company study cannot: JNJ's Regulatory / Legal count (6 mentions) is markedly higher than every other filer-period's (1-2), consistent with a pharmaceutical company's heavier regulatory-approval and litigation exposure; DAL is the only filer in the set to mention Cybersecurity / Data Privacy at all, plausible for an airline given its dependence on reservation and operational-technology systems; and Foreign Exchange / Currency mentions scale roughly with how internationally exposed each business is (Apple highest at 3 mentions in both periods; Etsy, DAL, and Costco's 10-Q lowest at 1). None of this is a claim the pipeline understands sector risk — it is evidence that risk language extracted independently, company by company, aggregates into a pattern that matches independently known facts about each business, the same kind of corroboration section 4.3 already used within Apple's own two periods, now demonstrated across companies as well.

CHAPTER 5: RECOMMENDATIONS AND CONCLUSION

5.1 Implementation Challenges and Caveats

Several practical challenges shaped this project as much as any deliberate design decision did. The largest was that SEC filers do not follow one consistent structural convention despite filing under the same regulatory form: the title-validated regex boundaries in segment_filing.py had to be hand-tuned against Apple's specific filing layout after a naive line-anchored match was found to land inside running page headers on other filers' documents, and this tuning does not automatically transfer to a company whose filings are formatted differently, a caveat repeated throughout Chapters 2 and 3 precisely because it is the pipeline's most significant real limitation.

A second challenge was calibrating the extraction constraints correctly: an instruction loose enough to let the model report a value at all had to be balanced against one strict enough to catch a hallucinated figure, and the fifteen-word source-snippet requirement was arrived at through iteration rather than being obviously correct from the outset. The Total Liabilities near-miss documented in Chapter 4, observed across three successive model/prompt states of this project (FY2024, then FY2023, then FY2024 again), where the model attached a fabricated dollar sign to an otherwise correct number, shows this balance is still imperfect even after repeated iteration. A related, unanticipated challenge, discovered only because a live run happened to be captured twice, was that GPT-4o's output is not perfectly reproducible across separate API calls even at a fixed sampling temperature of zero, which meant the project could not treat a single favorable evaluation run as a permanent guarantee of the pipeline's behavior.

A third set of challenges was more operational than analytical: every SEC EDGAR request needed a compliant, descriptive User-Agent header and careful accession-number resolution to avoid silently downloading the wrong filing, every OpenAI API call carried a real per-token cost that constrained how much filing text could reasonably be sent in one prompt, and Windows console encoding required explicit handling to avoid the pipeline crashing on ordinary filing text containing characters such as em dashes. None of these individually threatened the project's core claims, but together they consumed a disproportionate share of implementation effort relative to the extraction logic itself, and are worth naming explicitly so a future implementer does not underestimate the plumbing a filing-analysis pipeline like this one actually requires.

5.2 Proposed Solutions

The most immediate recommendation follows directly from the run-to-run variability documented in Chapter 4: before this pipeline is trusted for any repeated production use, it should run each extraction twice against the same filing section and treat a disagreement between the two runs as an automatic review flag, in addition to the existing snippet-verification check. The second live run captured in Figure 4.10 returned every citation as verified while the first run, on the same nominal period, flagged four as unverified, and neither run's confidence about itself was a reliable signal of which one, if either, was fully correct; a lightweight two-pass consistency check would catch exactly this kind of disagreement automatically, without requiring a human reviewer to happen to run the pipeline twice by chance the way this project did.

A second solution is to widen the ground truth before widening the company universe. The evaluation in Chapter 4 is convincing precisely because it was checked against sixteen manually verified metric-period observations for one company; scaling the pipeline to dozens of filers without a proportionally larger, independently built ground truth would mean scaling the claim of trustworthiness faster than the evidence supporting it. The recommended path is to add three to five additional filers from different sectors, in particular at least one smaller-capitalization company whose filings are less rigorously formatted than Apple's, and to build ground truth for each before drawing any conclusion about how well the segmentation and extraction logic generalizes beyond a single, well-structured filer. Section 4.4 documents this being carried out: four additional filers, each with its own independently built ground truth, were added and evaluated before any generalization claim was revised.

A third solution addresses the one concrete failure mode this project actually observed: the recurring Total Liabilities citation issue this project observed across three successive model/prompt states (FY2024, then FY2023, then FY2024 again), where the model attached a fabricated dollar sign to an otherwise correct number. Rather than treating every unverified citation as equally suspect, the verification step could be extended to distinguish a near-miss, where the claimed snippet differs from the source text by one or two inserted or dropped characters, from a genuine fabrication, where no similar text exists anywhere in the section at all. A simple edit-distance threshold on top of the existing exact-substring check would let a reviewer triage unverified citations by severity instead of treating all of them as equally uncertain.

A fourth, complementary solution is to cross-check the LLM-extracted figures against the same filing's XBRL-tagged structured data, which SEC filers are separately required to submit alongside the human-readable HTML. XBRL tags are machine-readable by design and were not used anywhere in this project's pipeline, but they represent an independent, non-LLM source that could validate GPT-4o's extracted values programmatically, effectively adding a second, deterministic verification layer alongside the existing snippet check, and would be a natural next module to add to the src/ directory.

A fifth, more operational recommendation is to add response caching keyed on the hash of the segmented section text, so that re-running the pipeline against a filing that has already been processed, whether to regenerate a chart, retry a failed step, or simply re-review a prior result, does not silently re-bill the OpenAI API for an identical extraction call. This is a small implementation change relative to the others, but it directly controls the cost constraint already identified in Chapter 3 and makes the two-pass consistency check proposed above materially cheaper to run routinely rather than only occasionally.

5.3 Implications for the Organization

For a research or analyst team, the most direct implication is where this pipeline belongs in an existing workflow: a first-pass extraction and triage layer that runs before a human opens the filing, not a replacement for reading it. Used this way, it turns the thirty to sixty minutes an analyst typically spends locating and transcribing figures and skimming for risk language into a five-minute review of a pre-populated table, attention directed at whatever the verification step has flagged. Chapter 4's accuracy results suggest this reallocation is defensible for a well-structured filer like Apple; the flagged Total Liabilities citation and the run-to-run variability in Figure 4.10 are reasons it should stop short of full automation.

The potential impact scales with how many filings an organization processes repeatedly. A single analyst reviewing one company's filings may see little benefit; a team covering thirty to fifty companies across a sector, each filing several times a year, faces a genuinely large repeated-extraction burden, and at that scale even a partial time reduction compounds into a meaningful capacity gain, freeing analyst time for the comparative, judgment-based work the pipeline does not attempt itself.

There is also a compliance and audit-trail implication that follows from a design choice rather than an incidental feature: because every metric and risk excerpt carries a verbatim citation and a computed verification flag, the output is inherently more auditable than a summary produced

from memory, letting a reviewer spot-check any figure in seconds. Organizations in regulated research contexts, where showing provenance matters as much as the number itself, gain more from this than from extraction speed alone.

The clearest risk-management implication is what Chapter 4 surfaces directly: output quality is not a fixed, one-time-proven property but something that can vary run to run on the same input, so adoption needs a standing verification habit, not a one-time validation exercise. A team that validates once and then trusts all subsequent output without re-checking verification flags would be relying on a guarantee this project's own evidence does not support.

5.4 Conclusion and Future Scope

This project set out to answer a specific question: can a general-purpose large language model, constrained by a narrow extraction schema and checked by independent, code-based verification, extract standardized financial metrics and risk signals from real SEC filings reliably enough to reduce an analyst's manual workload without introducing unacceptable hallucination risk. Across Apple's FY2023 and FY2024 10-K filings, the pipeline extracted all sixteen target metric-period values with one hundred percent accuracy against a manually built ground truth, and its mechanical snippet-verification step correctly caught the one citation, out of those sixteen, that was not a faithful quote of its source text, even though the underlying number in that case was correct. Both results, taken together rather than in isolation, support the alternative hypothesis set out in Chapter 2: the approach is accurate enough to meaningfully reduce manual verification effort, and its verification mechanism is doing real, demonstrable work rather than existing only as a theoretical safeguard.

The project's second, less anticipated finding, surfaced only because a live-run screenshot happened to be captured twice against the same nominal filing period, is arguably just as important as the headline accuracy number: a hosted commercial model's output is not perfectly reproducible from one API call to the next even with sampling temperature fixed to zero, which means a single favorable evaluation run is not a permanent guarantee of future behavior. This is precisely why the pipeline was built around per-run, per-citation verification rather than a one-time accuracy benchmark, and why that design choice, more than the accuracy percentage itself, is this project's most transferable contribution to how LLM-based extraction systems should be evaluated in other high-stakes, document-heavy domains.

The scope of what was validated here was intentionally narrow at first, and the immediate next step taken was concrete rather than speculative: section 4.4 validates the same, unmodified

pipeline against four additional filers spanning three further sectors and company sizes, each scored against its own independently built ground truth, closing the single most immediate gap this report originally flagged. What remains open is now correspondingly narrower. Implementing the two-pass consistency check proposed in section 5.1 would convert the run-to-run variability this project discovered, both within Apple's own live-run tests and, differently, across model snapshots in section 3.2, into a deliberate, always-on safeguard rather than something noticed only because a result happened to be captured twice. Adding XBRL cross-validation as a second, non-LLM verification layer alongside the existing snippet check would let the pipeline flag disagreement between two independent evidence sources rather than relying on the filing's narrative text alone — particularly relevant given that two of the three real failure modes section 4.4 found were numeric-value errors an independent, machine-readable cross-check would likely have caught immediately. Closing the one still-unresolved citation pattern documented in section 4.4, the fabricated dollar sign that survived a direct, explicit prompt instruction against it, would need either the edit-distance triage proposed in section 5.2 or acceptance that some fraction of citations will always need mechanical, not prompted, correction. Finally, expanding beyond five companies and eight metrics remains appropriate only once evaluation discipline keeps pace with scope, the same caveat this report opened with and still closes with.

Taken as a whole, this project demonstrates that the gap between an impressive large language model demonstration and a system a financial analyst could actually rely on is a gap that can be closed, not through a more powerful model, but through disciplined scope, mechanical verification of every claim the model makes, and an evaluation practice that reports failure alongside success. The eight-metric, single-company, two-period case study documented in this report is small by design; what it shows, within that deliberately narrow scope, is that the discipline works.

REFERENCES, DATA SOURCES AND ACKNOWLEDGMENTS

Data Sources

All filing text analyzed in this report was sourced directly from the U.S. Securities and Exchange Commission's EDGAR system (sec.gov / data.sec.gov), specifically Apple Inc.'s (ticker AAPL, CIK 0000320193) Form 10-K annual reports for fiscal years 2023, 2024, and 2025, and its Form 10-Q quarterly report for the period ended June 27, 2026; and, for the multi-company extension in section 4.4, the most recent Form 10-K on record as of testing for Johnson & Johnson (CIK 0000200406), Costco Wholesale Corporation (CIK 0000909832, plus its most recent Form 10-Q), Delta Air Lines (CIK 0000027904), and Etsy, Inc. (CIK 0001370637) — all retrieved via EDGAR's public submissions API and full-text filing archive. The extraction and risk-classification model used throughout, GPT-4o, was accessed through OpenAI's commercial API. The ground-truth values used to score extraction accuracy in Chapter 4 were constructed manually by the author, by directly reading the cited filings; no third-party financial database or pre-aggregated dataset was used at any stage.

Software and Open-Source Libraries

The pipeline is built on the Python scientific and web ecosystem and depends on the following open-source, third-party packages, each used under its own license and none authored as part of this project: sec-edgar-downloader and requests for EDGAR access; beautifulsoup4 and lxml for HTML parsing; the openai client library for model access; pandas for tabular data handling; plotly and matplotlib for visualization; streamlit for the interactive review application; tenacity for retry logic; python-dotenv for configuration; and pyrate-limiter for EDGAR rate-limit compliance. Full version constraints are listed in requirements.txt.

Code Authorship Declaration

The pipeline design, the project's requirements, and its evaluation methodology are the author's own. The author used Claude, Anthropic's AI coding assistant, as a development tool to help write and iteratively modify the implementation in src/ and app/ to meet this project's specific requirements: the fixed eight-metric and eight-risk-category taxonomies, the mechanical snippet-verification step described in Chapter 3, the period-comparison edge-case handling in compare_periods.py, and the ground-truth evaluation methodology in evaluate.py. No code in

this repository is presented as solely unassisted, hand-typed invention; the project's direction, design decisions, and the empirical evaluation applied to the resulting pipeline in Chapter 4 are the author's own contribution, with Claude used throughout as an AI-assisted coding tool rather than as the source of the project's ideas.

References

The following studies are cited in the literature review in Chapter 2, section 2.1.

Araci, D. (2019). FinBERT: Financial sentiment analysis with pre-trained language models. arXiv:1908.10063.

Chen, Z., Chen, W., Smiley, C., Shah, S., Borova, I., Langdon, D., Moussa, R., Beane, M., Huang, T.-H., Routledge, B., & Wang, W. Y. (2021). FinQA: A dataset of numerical reasoning over financial data. Proceedings of EMNLP 2021.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. ACM Computing Surveys, 55(12), 1-38.

Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., Dodds, Z. H., DasSarma, N., Tran-Johnson, E., et al. (2022). Language models (mostly) know what they know. arXiv:2207.05221.

Kogan, S., Levin, D., Routledge, B. R., Sagi, J. S., & Smith, N. A. (2009). Predicting risk from financial reports with regression. Proceedings of NAACL-HLT 2009.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Proceedings of NeurIPS 2020.

Li, F. (2010). The information content of forward-looking statements in corporate filings — A naive Bayesian machine learning approach. Journal of Accounting Research, 48(5), 1049-1102.

Loughran, T., & McDonald, B. (2011). When is a liability not a liability? Textual analysis, dictionaries, and 10-Ks. The Journal of Finance, 66(1), 35-65.

Loukas, L., Fergadiotis, M., Chalkidis, I., Spyropoulou, E., Malakasiotis, P., Androutsopoulos, I., & Paliouras, G. (2021). EDGAR-CORPUS: Billions of tokens make the world go round. Proceedings of the Third Workshop on Economics and Natural Language Processing.

Loukas, L., Fergadiotis, M., Androutsopoulos, I., & Malakasiotis, P. (2022). FiNER: Financial numeric entity recognition for XBRL tagging. Proceedings of ACL 2022.

Malo, P., Sinha, A., Korhonen, P., Wallenius, J., & Takala, P. (2014). Good debt or bad debt: Detecting semantic orientations in economic texts. Journal of the Association for Information Science and Technology, 65(4), 782-796.

Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W.-t., Koh, P. W., Iyyer, M., Zettlemoyer, L., & Hajishirzi, H. (2023). FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. Proceedings of EMNLP 2023.

Wu, S., Irsoy, O., Lu, S., Dabravolski, V., Dredze, M., Gehrmann, S., Kambadur, P., Rosenberg, D., & Mann, G. (2023). BloombergGPT: A large language model for finance. arXiv:2303.17564.

Xie, Q., Han, W., Zhang, X., Lai, Y., Peng, M., Lopez-Lira, A., & Huang, J. (2023). PIXIU: A large language model, instruction data and evaluation benchmark for finance. Proceedings of NeurIPS 2023 Datasets and Benchmarks Track.

Yang, Y., UY, M. C. S., & Huang, A. (2020). FinBERT: A pretrained language model for financial communications. arXiv:2006.08097.

Zhu, F., Lei, W., Huang, Y., Wang, C., Zhang, S., Lv, J., Feng, F., & Chua, T.-S. (2021). TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. Proceedings of ACL-IJCNLP 2021.